SG-Reg: Generalizable and Efficient Scene Graph Registration

Chuhao Liu, Zhijian Qiao, Jieqi Shi, Ke Wang, Peize Liu, and Shaojie Shen

***Abstract*—This article addresses the challenges of registering two rigid semantic scene graphs, an essential capability when an autonomous agent needs to register its map against a remote agent, or against a prior map. The handcrafted descriptors in classical semantic-aided registration, or the ground-truth annotation reliance in learning-based scene graph registration, impede their application in practical real-world environments. To address the challenges, we design a scene graph network to encode multiple modalities of semantic nodes: open-set semantic feature, local topology with spatial awareness, and shape feature. These modalities are fused to create compact semantic node features. The matching layers then search for correspondences in a coarse-to-fine manner. In the back end, we employ a robust pose estimator to decide transformation according to the correspondences. We manage to maintain a sparse and hierarchical scene representation. Our approach demands fewer GPU resources and fewer communication bandwidth in multiagent tasks. Moreover, we design a new data generation approach using vision foundation models and a semantic mapping module to reconstruct semantic scene graphs. It differs significantly from previous works, which rely on ground-truth semantic annotations to generate data. We validate our method in a two-agent simultaneous localization and mapping benchmark. It significantly outperforms the handcrafted baseline in terms of registration success rate. Compared to visual loop closure networks, our method achieves a slightly higher registration recall while requiring only 52 kB of communication bandwidth for each query frame.**

***Index Terms*—Deep learning in robotics and automation, multirobot systems, semantic scene understanding, simultaneous localization and mapping (SLAM).**

I. Introduction

GLOBAL registration in dense indoor scenes is crucial for visual simultaneous localization and mapping (SLAM) systems. It plays a key role in the performance of multiagent SLAM [1], multisession SLAM [2], [3], and long-term SLAM [4] systems. Relying on image matching [5], [6], [7] for loop closure detection and global registration is challenging because image matching is sensitive to viewpoint differences. Furthermore, in multiagent SLAM, broadcasting image data demand significant communication bandwidth. To address the limitations of visual-based loop closure detection, semantic-aided registration has been proposed [8], [9], [10], offering stronger viewpoint invariance and a more data-efficient representation, thereby reducing communication bandwidth.

Early works in semantic-aided registration explicitly maintain the semantic objects. To match reobserved semantic objects, they design handcrafted descriptors and similarity metrics based on the 2-D image feature [9], projected bounding box [8], [12], quadratic representation [13], or graph topology [10], [14], [15]. Using the calculated similarity metric, they identify matched nodes by either setting a threshold or designing a set of verification rules. Although these works introduced pioneering SLAM frameworks that incorporate semantic landmarks, they often face long-tailed cases in real-world experiments due to the noise commonly present in the reconstructed semantic objects.

In recent years, learning-based semantic scene graph registration methods [16], [17] have emerged. These methods encode the semantic scene graph and determine multiple levels of correspondences by solving partial assignment problems. They eliminate the need for handcrafted similarity metrics between semantic nodes. However, the current scene graph learning methods are trained on datasets with ground-truth annotations, resulting in a significant domain gap between the semantic scene graphs from the annotated dataset and those in the real-world scenarios, especially when noise is present. As a result, these learning-based methods face challenges in generalization and demonstrate limited effectiveness in real-world experiments.

Such real-world semantic noise and sensor noise motivate us to propose a new data generation method. We reconstruct the semantic scene graphs using three vision foundation models [18], [19], [20] and a semantic mapping module: FM-Fusion [11]. In real-world indoor environments, the reconstructed semantic scene graphs exhibit higher quality compared to those generated using Mask R-CNN [21] combined with earlier semantic mapping methods [22], [23] . Although our method effectively limits the semantic noise to appropriate levels, it cannot completely eliminate the noise. For example, as shown in Fig. 1, the zoomed subvolumes from the two scene graphs reveal semantic inconsistencies due to noise in real-world reconstructions. In contrast to previous scene graph learning methods [16], [17], which rely on ground-truth semantic annotations to generate data, our approach introduces a new and practical challenge.

Received 27 February 2025; accepted 19 April 2025. Date of publication 5 June 2025; date of current version 24 June 2025. This work was supported by the HKUST-DJI Joint Innovation Laboratory. This article was recommended for publication by Guest Associate Editor G. Cioffi and Editor J. Civera upon evaluation of the reviewers' comments. *(Corresponding author: Jieqi Shi.)*

Chuhao Liu, Zhijian Qiao, Peize Liu, and Shaojie Shen are with the Department of Electronic and Computer Engineering, The Hong Kong University of Science and Technology,, Hong Kong (e-mail: cliuci@connect.ust.hk; zqiaoac@connect.ust.hk; pliuan@connect.ust.hk; eeshaojie@ust.hk).

Jieqi Shi is with the School of Intelligence Science and Technology, Nanjing University, Nanjing 215163, China (e-mail: isjieqi@nju.edu.cn).

Ke Wang is with the School of Information Engineering, Chang'an University, Xi'an 710064, China (e-mail: kwangdd@chd.edu.cn).

Code is available online at http://github.com/HKUST-Aerial-Robotics/SG-Reg.

Digital Object Identifier 10.1109/TRO.2025.3577020

1941-0468 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

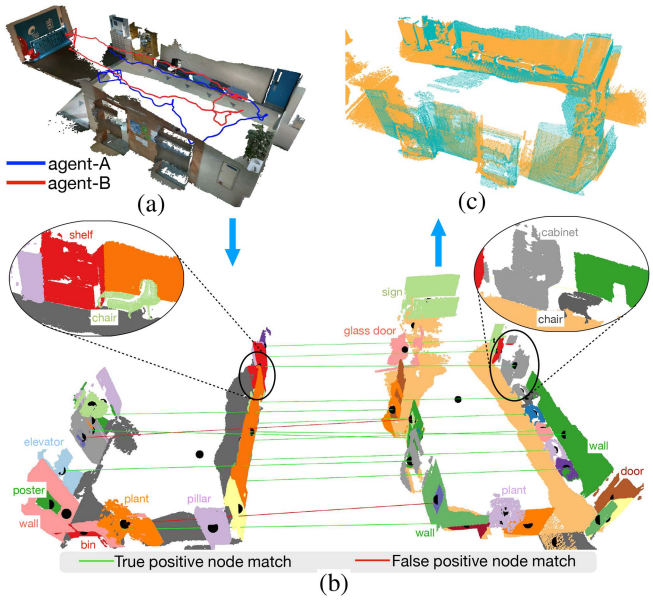


Fig. 1. Register the semantic scene graphs in the two-agent SLAM system. (a) Captured RGB-D sequences from the two agents in a real-world indoor scene. The two agents move in an opposite direction, creating a large viewpoint difference between their cameras. (b) Visualization of the matched nodes between the semantic scene graphs, which are from the two agents. The scene graphs are constructed using FM-Fusion [11]. The zoomed subvolume showcases examples of inconsistent semantic nodes. For better visualization, only a subset of the semantic labels is displayed. (c) Registration result.

To address the challenges of noisy scene graphs and achieve generalizable registration, we propose encoding multiple types of information in the scene graph: semantic label, local topology, and geometric shape. The semantic label is encoded by the pretrained BERT [24] model, which is known for its strong generalization capabilities and naturally accepts open-set labels. In learning the local topology of a semantic node, we introduce a triplet descriptor and integrate it into the graph neural network (GNN) [25] to boost the spatial awareness of the learned feature. It enhances the descriptive power of the local topology while ensuring invariance to a global transformation in four degrees of freedom (DoFs). For shape information, the shape network reads the point cloud of each semantic node and aggregates a geometric shape feature. We fuse all the information to generate a comprehensive semantic node feature. The scene graph represents the original dense 3-D point cloud into sparse object-level features, thus enabling the network layers to process the whole scene while demanding fewer GPU resources. The last module of our network consists of two levels of graph-matching layers: one for matching semantic nodes and another for matching their point clouds. Correspondences at each level are efficiently determined using an optimal transport algorithm [26].

The back end reads the point correspondences. It prunes outliers via maximum clique (MAC) [27] and estimates pose through the robust estimator G3Reg [28]. The back end is designed to tolerate a certain degree of point outliers, thereby further enhancing the overall generalization ability in cross-domain experiments, where the outlier ratio tends to be relative high.

This work focuses on the semantic scene graph registration task in indoor visual-inertial SLAM conditions. We highlight the key contributions of our work as follows.

1) We introduce a learning-based semantic scene graph registration approach. Our approach employs a triplet-boosted GNN layer, which is designed to capture the semantic nodes' topological relationships along with spatial awareness. Compared to a vanilla GNN network, it offers a more descriptive representation of semantic nodes while maintaining invariance under global transformations in four DoFs.
2) The shape network generates a shape feature for each semantic node, which is then fused into the node's feature representation to improve scene graph matching performance. Thanks to the sparse nature of semantic nodes, the computational complexity and GPU memory consumption are significantly reduced.
3) We propose an automatic data generation method for scene graph learning. We utilize the vision foundation models [18], [19], [20] and a semantic mapping [11] module to construct semantic scene graphs. This innovative data generation approach enables training the network using posed RGB-D sequences, eliminating the reliance on ground-truth annotations. Furthermore, it significantly reduces the domain gap between the training data and real-world evaluation data.
4) We thoroughly evaluate our method against the baseline model [17] using real-world reconstructed scene graphs from 3RScan [29]. Our approach achieves significantly higher registration recall (RR) than SG-PGM [17], while requiring substantially less GPU resources.
5) We deploy our work in a two-agent SLAM system and register the two scene graphs in a coarse-to-fine paradigm, as shown in Fig. 1. With far lower communication bandwidth, our registration substantially outperforms handcrafted semantic descriptor [15] and achieves a success rate 0.7% higher than the combination of NetVLAD and LightGlue [5], [7], [30].

II. Related Works

A. Image Registration

The current state-of-the-art visual SLAM systems [2], [23], [31] rely on DBoW [32] to detect loop closure and register camera poses. In VINS-Mono [31], for example, if DBoW detects a loop closure candidate, it extracts binary robust independent elementary features (BRIEF) descriptors and performs brute-force matching between the looped images. Thereafter, VINS-Mono registers images using RANSAC-based perspective-n-point (PnP) [33]. To reject false loop closure or inaccurate registration, the visual SLAM systems employ comprehensive geometric verification in pose graphs [23], [31] or covisibility graphs [2].

In recent years, learning-based image matching has been proposed and primarily improves registration performance in visual SLAM. SuperGlue [6] and LightGlue [7] encode image features using attention layers and search feature correspondences. NetVLAD [5] is combined with them to detect loop closure in a coarse-to-fine paradigm [30], [34]. These learning-based methods significantly outperform the classical loop closure detection in visual SLAM. Nevertheless, in multiagent SLAM, the

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

learning-based image registration methods require considerable communication bandwidth. In addition, they often result in false registrations under significant viewpoint disparities.

B. Point Cloud Registration

Given that visual SLAM is capable of generating a dense point cloud map [35], it can be integrated with point cloud registration to compute a global transformation. Traditional methods for point cloud registration [36] involve extracting FPFH features [37] from the input clouds and solving the pose estimation problem using the Gaussian–Newton method. Subsequently, robust pose estimators [28], [38] were introduced to estimate poses even with a significant outlier presence.

Meanwhile, numerous learning-based point cloud registration (PCRs) [39], [40], [41] are proposed. They use 3-D convolution network [42] or PointNet [43] as a point cloud backbone, leveraging the extracted features to establish correspondences. They subsequently solve the relative pose in a closed-form using singular value decomposition (SVD) or weighted SVD [44]. Among the learning-based PCR, GeoTransformer [41] demonstrates promising registration performance in the 3DMatch dataset [45], which includes partial scans of point clouds in real-world indoor environments. Despite its superior capabilities, GeoTransformer encodes a superpoint feature and incorporates attention layers [46] on these superpoints. Due to the density of the superpoints, the attention operation demands substantial GPU resources, limiting its scalability in large-scale scenes.

C. Scene Graph Construction

In order to integrate a scene graph into visual SLAM, the initial step is to construct a scene graph. Kimera [23] and its series of works are pioneer works in this area. Using Mask R-CNN [21], Kimera incorporates semantic segmentation obtained from images into a metric-semantic map. This map is built upon the TSDF voxel grid map [47] and is integrated following SemanticFusion [22]. Then, Kimera clusters the dense metric-semantic map into hierarchical levels of representations: objects, places, floors, and buildings. Later, S-Graph [48] and S-Graph+[49] segment 3-D planes to build their scene graph, which is further integrated in an optimizable factor graph. The semantic scene graph closes the gap between robot perception and human perception. It is regarded as the spatial perception engine [50] for spatial intelligence in the future.

Since the release of vision foundation models, recent advancements in semantic mapping [51], [52] have increasingly incorporated these models to reconstruct 3-D semantic maps. Among these, FM-Fusion [11] focuses on RGB-D SLAM in indoor environments. It integrates object detections from RAM-Ground-SAM [18], [19], [20] to reconstruct an instance-aware semantic map. Like Fusion++ [9], FM-Fusion represents each semantic instance in a separate TSDF submap. It fuses semantic labels using a Bayesian filter while independently modeling the measurement likelihood from RAM [18] and GroundingDINO [19]. By leveraging vision foundation models, FM-Fusion has achieved more accurate semantic instance segmentation compared to Kimera on the ScanNet benchmark. Based on the semantic instances, we construct a higher quality semantic scene graph without relying on ground-truth annotations. However, RAM-Ground-SAM still predicts noisy instance segmentation and incorrect semantic labels. When compared to scene graphs generated using annotations in ScanNet or 3RScan, the reconstructed scene graphs using FM-Fusion exhibit significant noise.

D. Semantic Data Association

Data association between semantic representations is the most important step to guarantee a successful registration. In semantic SLAM works, The step relies on explicit representations. Bowman et al. [12] represent each object using a bounding box. If two object nodes belong to the same semantic category and their intersection over union (IoU) exceeds a predefined threshold, they are considered a match. Building on Kimera, Hydra [15] constructs handcrafted semantic descriptors by aggregating semantic histograms from nearby objects and computes node similarity using these descriptors. More recently, SlideSLAM [53] has determined object similarity based on identical semantic categories and similar bounding box shapes. To incorporate topological information into semantic data association, X-View [14] introduces a random walk descriptor (RWD) that records semantic labels along walking routes. The descriptor represents the topology information explicitly. When calculating the similarity metric, X-View counts the number of rows with identical RWDs to determine a similarity score. To enhance the spatial descriptiveness of RWD, Lin et al. [10] integrate distance information into the arrangement of RWDs. Similarly, Liu and Shen [55] utilize spatial priors to construct RWDs. To reduce false matches between nodes, some approaches [56] explicitly construct edge descriptors to verify geometric consistency among matched nodes. Along the same lines, Kabalar et al. [57] employ node triplets to validate the correctness of matched nodes.

The aforementioned semantic-related descriptors are explicitly represented through labels, bounding boxes, and topology, rather than being implicitly encoded from these attributes. They solved their targeting scenarios but faced corner cases in general real-world evaluation. For example, in the inconsistent scene graphs, as shown in Fig. 1, they may need to set a topology or edge threshold for each semantic category. The parameter tuning workload is huge.

Inspired by the learning-based graph matching models [58], we believe training a neural network to learn semantic data association is a promising direction. The latest version of Hydra [50] proposes encoding object nodes using a GNN, though it omits shape features. SGAligner [16] is the first work, to our knowledge, to focus on learning scene graph matching. It encodes multiple modalities from semantic nodes, including semantic labels, center positions, and relationship labels. Subsequently, SG-PGM enhances this approach by incorporating shape features into scene graph learning. It samples points from each semantic node and aggregates them using GATv2 [25]. SGAligner and SG-PGM adopted the geometric-related layers from GeoTransformer, including its superpoint matching, point matching, and local-to-global registration layer. Both of them are trained and evaluated on the 3RScan dataset [29], requiring ground-truth annotations to construct their scene graph.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF SEMANTIC DATA ASSOCIATION METHODS

	Year	Descriptor in matching nodes/objects			Dense Matching	Real-world Visual SLAM Experiment	
		Type	Topology	Shape		Semantic Annotation	LCD Baselines
Bowman *et al.* [12]	2017	Explicit	×	B-box	×	DPM	DBoW
Fusion++ [9]	2018	Explicit	×	×	2D Points	Mask R-CNN	-
X-View [14]	2018	Explicit	✓	×	×	SegNet	DBoW,NetVLAD
Lin *et al.* [10]	2021	Explicit	✓	B-box	2D Points	SOLOv2	DBoW+ORB
Kimera [23]	2021	Explicit	×	B-box	×	Mask R-CNN	-
Hydra [15]	2022	Explicit	✓	B-box	2D Points	Mask R-CNN	-
SlideSLAM [53]	2024	Explicit	×	B-box	×	YOLO	-
Found. SPR [50]	2023	Exp. & Imp.	✓	B-box	2D Points	Mask R-CNN	-
SGAligner [16]	2023	Implicit	✓	×	3D Points	Ground-truth	N/A
SG-PGM [17]	2024	Implicit	✓	Point Cloud	3D Points	Ground-truth	N/A
Living Scenes [54]	2024	Implicit	✓	Point Cloud	3D Points	Ground-truth	N/A
SG-Reg (Ours)	2024	Implicit	✓	Point Cloud	3D Points	RAM-Ground-SAM	NetVLAD+LightGlue

It covers the loop closure detection module in semantic SLAM and graph match modules in scene graph registration. The methods based on implicit feature descriptor have incorporated neural networks to predict the data association. The B-box refers to a bounding box and the LCD refers to loop closure detection.

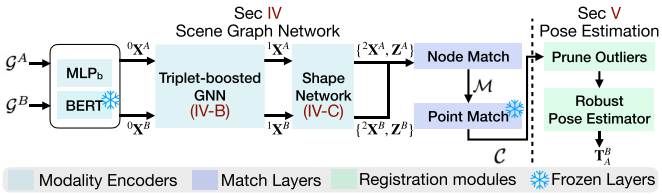


Fig. 2. Our system overview. We denote the encoded node features as ${}^{l}\mathbf{X}^{A/B}$, where its layer index $l \in \{0, 1, 2\}$.

We provide a comprehensive summary of the semantic data association methods in Table I. Our approach is fundamentally different from previous semantic SLAM works, which reconstruct explicit semantic representations and create handcrafted descriptors. In contrast, we learn to encode scene graphs. The work most similar to ours is SG-PGM, but we have three core differences. First, our triplet-boosted GNN encodes semantic nodes with enhanced spatial awareness, outperforming the vanilla GAT used in SG-PGM. Second, we avoid aggregating point cloud features through attention layers, significantly reducing GPU memory usage and speeding up inference. Third, we evaluate our method in real-world SLAM experiments, whereas SG-PGM and prior learning-based works only assess performance using scene graphs with ground-truth annotations. We integrate the data generation process with semantic mapping [11], allowing us to train and evaluate our network on scene graphs constructed from semantic mapping.

III. PRELIMINARY

As shown in Fig. 2, SG-Reg is composed of a scene graph network and a robust pose estimator. The scene graph network reads a pair of semantic scene graphs, $\{\mathcal{G}^A, \mathcal{G}^B\}$, as input and produces node matches $\mathcal{M}$ along with point correspondences $\mathcal{C}$. Using these hierarchical matches, the pose estimator calculates a relative transformation $\mathbf{T}_A^B$ between the two scene graphs. In the following, we summarize the scene representations used in our approach.

A. Explicit Representation

We denote a semantic scene graph as $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V}$ is the node set and $\mathcal{E}$ is the edge set. A semantic node $\mathbf{v}_i$ has the following attributes.

1) s is its open-set semantic label in text format.
2) $\mathbf{b} \in \mathbb{R}^3$ represents the bounding box length, width, and height.
3) $\mathbf{o} \in \mathbb{R}^3$ is its geometric center.
4) $\mathbf{P}$ is its point cloud.

B. Implicit Representation

It incorporates multiple modalities:

1) shape feature $\mathbf{f}_i \in \mathbb{R}^{d_s}$, where d_s is the dimension of the shape feature;
2) the node feature ${}^{l}\mathbf{x}_i$, where l is the layer index. Before the shape network, the node feature is in the dimension ${}^{0,1}\mathbf{x}_i \in \mathbb{R}^d$. After the fusion with shape features, the final node feature is in the dimension ${}^{2}\mathbf{x}_i \in \mathbb{R}^{d+d_s}$;
3) point features $\mathbf{z}_i \in \mathbb{R}^{K_p \times d_z}$, where K_p is the number of sampled points and d_z is the dimension of the point feature;
4) we use small symbols to denote features from one node and capital symbols to denote features from a scene graph. For example, ${}^{2}\mathbf{x}_i$ is a node feature, and ${}^{2}\mathbf{X}^A$ is the stacked node feature from $\mathcal{G}^A$.

C. Node Index

For a pair of scene graphs $\{\mathcal{G}^A, \mathcal{G}^B\}$, we denote their node indices $\mathcal{A} = [1, \ldots, |\mathcal{V}^A|]$ and $\mathcal{B} = [1, \ldots, |\mathcal{V}^B|]$.

D. Data Association

The scene graph network generates node matches $\mathcal{M}$ and point matches $\mathcal{C}$.

1) A node match $m = \{i, j, \omega_n\}$, where $m \in \mathcal{M}$, consists of the associated node indices $i \in \mathcal{A}, j \in \mathcal{B}$ and the node confidence $\omega_n \in [0, 1]$.
2) A global set of point correspondences $\mathcal{C} = \{(\mathbf{p}_k, \mathbf{q}_k)\}$, where $\mathbf{p}_k, \mathbf{q}_k \in \mathbb{R}^3$.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

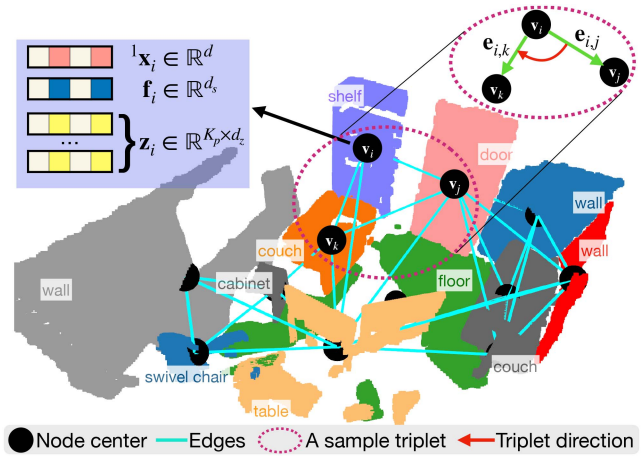


Fig. 3. Visualization of a semantic scene graph from ScanNet *scene0025_00*. Each node's point cloud is distinctly colored. For node $\mathbf{v}_i$, we illustrate one of its triplet. In addition, the implicit features derived from $\mathbf{v_i}$ are displayed.

E. Transformation

The relative transformation from $\mathcal{G}^A$ to $\mathcal{G}^B$ is denoted as $\mathbf{T}_A^B \in \mathrm{SE}(3)$.

IV. Scene Graph Network

A. Semantic Scene Graph Construction

The semantic scene graph is constructed using FM-Fusion [11]. We extract its semantic instances and construct semantic nodes $\mathcal{V}$ in the scene graph. Those instances that are too small or have insufficient observations are excluded. For a node $\mathbf{v}$, its semantic label s comes from the prediction from FM-Fusion. The node can be observed multiple times by GroundingDINO, and FM-Fusion fuses multiple label measurements to predict a final label s. The point cloud $\mathbf{P}$ is extracted from the instancewise submap via 3-D interpolation. The interpolation process is similar to marching cube [59], but it only interpolates points that have three neighboring voxels. We extract the node center $\mathbf{o}$ from $\mathbf{P}$. Then, we calculate the minimum bounding box from $\mathbf{P}$ using O'Rourke's algorithm [60]. The bounding box shape $\mathbf{b} \in \mathbb{R}^3$ is recorded, representing its length, width, and height. We ignore the orientation from each semantic node to minimum additional noise. Finally, a semantic node is constructed $\mathbf{v} = \{s, \mathbf{b}, \mathbf{o}, \mathbf{P}\}$. We construct the representations for other nodes and generate $\mathcal{V}$.

Once the node set $\mathcal{V}$ is constructed, we compute the distance between each pair of nodes. The distance can be calculated from the node's center. If their distance is less than a threshold τ_d, we connect an edge between them. The distance threshold τ_d is decided by node's shape size, meaning that larger nodes connect to more distant nodes and a greater number of nodes. We record the generated edge set $\mathcal{E}$. As shown in Fig. 3, we visualize an example of scene graph $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$.

The input semantic scene graphs $\{\mathcal{G}^A, \mathcal{G}^B\}$ have a relative transformation in four DoFs, including 3-D position and relative yaw rotation. Relative rotation in roll and pitch angle is already estimated by visual-inertial SLAM [31] accurately.

B. Triplet-Boosted GNN

First, we initialize the input features for the GNN module. We run BERT [24] to encode semantic labels and run a single-layer MLP to encode bounding box size. A node $\mathbf{v}_i \in \mathcal{V}^A$ has its node feature initialized as follows:

$$^0\mathbf{x}_i = [\mathrm{BERT}(s_i) \| \mathrm{MLP}_b(\mathbf{b}_i)] \tag{1}$$

where $^0\mathbf{x}_i \in \mathbb{R}^d$ and $[\cdot\|\cdot]$ denotes a concatenation.

The current attentional GNN incorporates a relative position encoding (RPE) in its self-attention aggregation [7]

$$\mathrm{RPE}(\mathbf{o}_i, \mathbf{o}_j) = \Phi^R(\mathbf{o}_i - \mathbf{o}_j) \tag{2}$$

where $\mathbf{o}_{i/j}$ are the nodes centers and Φ^R is a rotary encoding. RPE has been shown to enhance learning in vision tasks [7], [61] and language tasks [24]. However, RPE exhibits a critical limitation in 3-D geometric tasks: it is variant to yaw-angle rotations. Specifically, when the input graphs are transformed by a yaw-angle rotation, RPE fails to produce rotation-invariant features, which are essential for robust matching in such scenarios.

To address the challenge, we design a triplet descriptor to enhance GNN learning. The triplet descriptor is designed to be invariant to a global transformation in four DoFs. As shown in Fig. 3, we set a semantic node $\mathbf{v}_i \in \mathcal{V}^A$ as the anchor of a triplet and randomly sample two of its neighbor nodes $\{\mathbf{v}_j, \mathbf{v}_k\}$ to be the corners of the triplet. The triplet feature is $\mathbf{t}_{j,k}^i$. In computing $\mathbf{t}_{j,k}^i$, we keep the corner nodes $\mathbf{v}_j$ and $\mathbf{v}_k$ in an anticlockwise order along the z-axis in a visual SLAM coordinate [31].

Mathematically, the triplet feature is defined as follows:

$$\mathbf{t}_{j,k}^i = \begin{cases} \mathbf{t}_{[j,k]}^i, & \text{if } (\mathbf{e}_{i,j} \times \mathbf{e}_{i,k})_z \geq 0 \\ \mathbf{t}_{[k,j]}^i, & \text{if } (\mathbf{e}_{i,j} \times \mathbf{e}_{i,k})_z < 0 \end{cases} \tag{3}$$

where the relative position vector $\mathbf{e}_{i,j} = \mathbf{o}_j - \mathbf{o}_i$, and $(\cdot \times \cdot)_z$ is the z-axis value after a cross product. Then, an ordered triplet feature is initialized

$$\mathbf{t}_{[j,k]}^i = [^0\mathbf{x}_j \| ^0\mathbf{x}_k \| \mathbf{g}_{j,k}^i]. \tag{4}$$

$\mathbf{g}_{j,k}^i$ is a geometric embedding

$$\mathbf{g}_{j,k}^i = [\psi^L(|\mathbf{e}_{i,j}|) \| \psi^L(|\mathbf{e}_{i,k}|) \| \psi^A(\hat{\mathbf{e}}_{i,j} \cdot \hat{\mathbf{e}}_{i,k})] \tag{5}$$

where the length of triplet edges $|\mathbf{e}_{i,j}|, |\mathbf{e}_{i,k}| \in \mathbb{R}$ and $\hat{\mathbf{e}}_{i,j}$ and $\hat{\mathbf{e}}_{i,k}$ are the normalized vectors. The term $(\hat{\mathbf{e}}_{i,j} \cdot \hat{\mathbf{e}}_{i,k})$ represents cosine value of the triplet angle. Sinusoidal functions ψ^L and ψ^A are applied to embed the edge length and the triplet angle. The sinusoidal functions have been used in GeoTransformer [41] to encode their geometric scalars.

Now that we have illustrated the creation of triplet features, we explain how to integrate them into a GNN. The triplet-boosted GNN reads the initialized node features $^0\mathbf{X}^A$ and outputs them as $^1\mathbf{X}^A$. The residual message passing update for all i in A is

$$^1\mathbf{x}_i = {}^0\mathbf{x}_i + \mathrm{MLP}[^0\mathbf{x}_i \| \mathbf{m}_i]. \tag{6}$$

To compute the message feature $\mathbf{m}_i$, we first sample a set of triplets from its associated edge set $\mathcal{E}(i, \cdot)$. The feature is then computed using an attention mechanism [46], which aggregates

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

the embeddings of all sampled triplets

$$\mathbf{m}_i = \sum_{(j,k)\in\mathcal{E}(i,\cdot),j\neq k} \alpha^i_{j,k}\left(\mathbf{W}^V(\mathbf{t}^i_{j,k})\right). \tag{7}$$

The attention score $\alpha^i_{j,k}$ is computed by a softmax over all of the query–key similarities:

$$\alpha^i_{j,k} = \text{softmax}_i\left(\mathbf{W}^Q(\mathbf{t}^i_{j,k})\left(\mathbf{W}^K(\mathbf{t}^i_{j,k})\right)^T\right) \tag{8}$$

where $\mathbf{W}^K$, $\mathbf{W}^Q$, and $\mathbf{W}^V$ are projection encoders before the attention layer. This step leverages the attention mechanism to aggregate relevant triplets associated with each node.

We follow (6) to encode all the nodes in $\mathcal{G}^A$ and $\mathcal{G}^B$. The outputs after the triplet-boosted GNN are

$$\{{}^1\mathbf{X}^A, {}^1\mathbf{X}^B \big| {}^1\mathbf{X}^A \in \mathbb{R}^{|\mathcal{A}|\times d}, {}^1\mathbf{X}^B \in \mathbb{R}^{|\mathcal{B}|\times d}\}. \tag{9}$$

Compared to the vanilla graph attention network [25], the triplet boosts the spatial awareness of the GNN. Compared to the GNN incorporated with RPE [7], our approach ensures that the features remain invariant to yaw rotation. In addition, our process aligns more closely with human intuition. Humans recognize a room layout by describing its object topology with spatial awareness. For example, a human may refer to a couch in his office facing a television and having a table on its left-hand side. Our triplet descriptor captures local topology with spatial awareness, mimicking human intuition. In the SLAM domain, similar triplet descriptors [62], [63], [64], [65] have been proposed, although they represent the triplet explicitly. We believe triplet is a relatively stable local structure in the cross-domain scenario we target. Even under noisy scene graphs, if a node feature aggregates one or two correct triplets, it is still highly likely to find the correct match.

C. Shape Network

The shape network generates geometric features at two levels. It includes a point backbone that learns point features and a shape backbone that learns the nodewise shape features.

Given a pair of scene graphs $\{\mathcal{G}^A, \mathcal{G}^B\}$, we extract their global point cloud $\{\mathcal{X}, \mathcal{Y}\}$, where each point is annotated with its parent node index. We downsample them into four resolutions $\{{}^l\mathcal{X}, {}^l\mathcal{Y}\}_l$, where the layer index $l = \{0, 1, 2, 3\}$. The parent node index is maintained during downsampling.

In the following steps, we explain encoding for $\{{}^l\mathcal{X}\}_l$ and it goes the same way for $\{{}^l\mathcal{Y}\}_l$. As shown in Fig. 4, the point backbone generates point features in multiple layers. We only take the second finest level of point features as a hidden state feature $\mathbf{H}^A$. We did not take the finest level of point cloud features, because they are redundant, as noted in previous work [41]. Utilizing the node indices maintained in ${}^1\mathcal{X}$, we can directly sample the nodewise points from $\mathbf{H}^A$. During sampling, we only keep K_p points for each node. If a node finds too few or too many points, we adjust the number to K_p through random sampling or zero padding. The sampled nodewise points and features are $\mathcal{P}^A$ and $\mathbf{Z}^A$.

The shape backbone has one layer of encoder and decoder. It aggregates point features following the annotated node indices in ${}^1\mathcal{X}$ and generates nodewise shape features $\mathbf{F}^A$. Its decoder

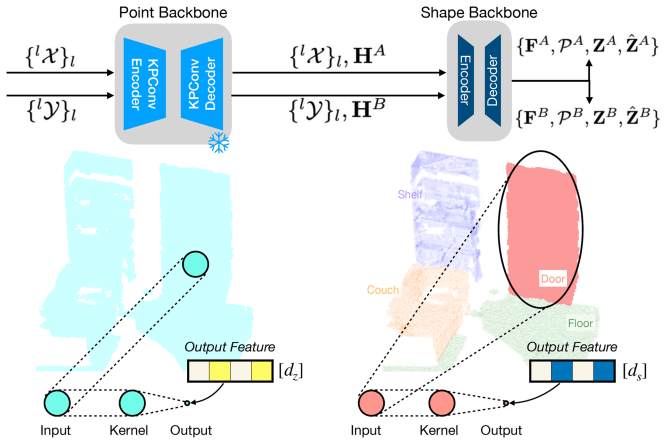


Fig. 4. Visualization of the shape network structure and its point aggregation kernels. Point backbone uses grid subsampling to decide aggregation kernels, which are small and dense. Shape backbone following instance segmentation to create aggregation kernels, which are large and sparse.

generates decoded point features $\hat{\mathbf{Z}}^A$, allowing dense supervision of the shape backbone during training.

We summarize the output representations of $\mathcal{G}^A$ as follows:

1) node shape feature $\mathbf{F}^A \in \mathbb{R}^{|\mathcal{A}|\times d_s}$;
2) node points $\mathcal{P}^A \in \mathbb{R}^{|\mathcal{A}|\times K_p\times 3}$;
3) node point feature $\mathbf{Z}^A \in \mathbb{R}^{|\mathcal{A}|\times K_p\times d_z}$;
4) decoded point feature $\hat{\mathbf{Z}}^A \in \mathbb{R}^{|\mathcal{A}|\times K_p\times d_z}$.

At the last step of the shape network, we fuse the shape features into the node features by concatenation

$${}^2\mathbf{X}^A = [{}^1\mathbf{X}^A \| \mathbf{F}^A],\ {}^2\mathbf{X}^A \in \mathbb{R}^{|\mathcal{A}|\times(d+d_s)}.$$

Notice that the shape fusion can run before the triplet-GNN or after the triplet-GNN. We investigate the influence of early fusion and late fusion in our experiment in Section VII-D3.

Some previous PCR works [39], [41] also employ a point cloud backbone to extract point cloud features. The uniqueness of our method stems from the addition of a shape backbone after the point backbone. This approach offers two key advantages. First, it reduces the density of the output features from the point backbone. In the previous point cloud learning methods [39], [41], they run multiple attention layers on the hidden state features after the point backbone. Due to the densely distributed hidden state features, their attention layers consume a huge GPU memory and can only run on small-scale scenes. Second, the shape backbone generates a shape feature to represent the geometric attribute of each semantic node, enabling the direct fusion of shape features with other modality features to create a compact semantic node feature.

D. Hierarchical Graph Match

With the learned node features and their point features, we can run hierarchical graph matching layers to search for correspondences. As shown in Table II, we summarize the node assignment matrix $\mathbf{A}^X$ and point assignment matrix $\mathbf{A}^Z$. The decoded point assignment matrix $\hat{\mathbf{A}}^Z$ is used to supervise the shape network training and is skipped in the inference.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

TABLE II
ASSIGNMENT MATRIX DIMENSIONS

	Input	Dimensions	Inf.
Node Assignment $\mathbf{A}^X$	$\{{}^l\mathbf{X}^A, {}^l\mathbf{X}^B\}$	$\lvert\mathcal{A}\rvert \times \lvert\mathcal{B}\rvert$	✓
Point Assignment $\mathbf{A}^Z$	$\{\mathcal{M}, \mathbf{Z}^A, \mathbf{Z}^B\}$	$\lvert\mathcal{M}\rvert \times K_p \times K_p$	✓
Point Assignment $\hat{\mathbf{A}}^Z$	$\{\bar{\mathcal{M}}, \hat{\mathbf{Z}}^A, \hat{\mathbf{Z}}^B\}$	$\lvert\bar{\mathcal{M}}\rvert \times K_p \times K_p$	×

The Inf. refers to the assignment computed at the inference stage. $\bar{\mathcal{M}}$ refers to the ground-truth node matching during the training.

Next, we illustrate the construction of the assignment matrix at each level. In previous work, SuperGlue [6] formulates the graph matching problem as a differential partial assignment problem. It determines the optimal match by the Sinkhorn algorithm [26]. We use similar techniques in graph matching.

At the node match layer, we calculate the node similarity matrix and the *node assignment matrix* as follows:

$$\mathbf{S}^X_{i,j} = \text{Linear}(\mathbf{x}_i)^T \text{Linear}(\mathbf{x}_j) \quad \forall (i,j) \in \mathcal{A} \times \mathcal{B} \tag{10}$$

$$\mathbf{A}^X_{i,j} = \underset{(k\in\mathcal{A})}{\text{Softmax}} \left(\mathbf{S}^X_{k,j}\right)_k \underset{k\in\mathcal{B}}{\text{Softmax}} \left(\mathbf{S}^X_{i,k}\right)_k. \tag{11}$$

We take dual normalization in computing $\mathbf{A}^X$ to suppress the negative match pairs. If an assignment score $\mathbf{A}^X_{i,j}$ is higher than a threshold and it is a mutual top-k score in $\mathbf{A}^X$, we extract it as a matched node pair $m = \{i, j, \omega\}$, where match confidence $\omega = \mathbf{A}^X_{i,j}$. Thus, we predict a set of node matches $\mathcal{M}$.

Guided by the node matches $\mathcal{M}$, the point matching layer searches for the point correspondences $\mathcal{C}$. Regarding a matched node pair (i, j), its point similarity matrix is calculated as follows:

$$\mathbf{S}^Z_{i,j} = (\mathbf{z}^A_i)^T \mathbf{z}^B_j, \mathbf{S}^Z_{i,j} \in \mathbb{R}^{K_p \times K_p} \tag{12}$$

Then, we apply the optimal transport algorithm [26] on $\mathbf{S}^Z_{i,j}$ to compute the *point assignment matrix* $\mathbf{A}^Z_{i,j} \in \mathbb{R}^{K_p \times K_p}$. The optimal transport algorithm can be seen as a differential version of the Hungarian algorithm [66]. It has been applied to solve the bipartite matching problem in SuperGlue [6] and GeoTransformer [41].

We select the mutual top-k pairs in $\mathbf{A}^Z_{i,j}$ as the point correspondences $\mathcal{C}_{i,j}$

$$\mathcal{C}_{i,j} = \{\mathbf{p}_k, \mathbf{q}_k | \mathbf{p}_k \in \mathcal{P}^A_i, \mathbf{q}_k \in \mathcal{P}^B_j, k \in \text{mtop-}k(\mathbf{A}^Z_{i,j})\} \tag{13}$$

where $\{\mathcal{P}^A_i, \mathcal{P}^B_j\}$ are the nodewise points, as illustrated in Section IV-C. Finally, we search point correspondences in all the matched nodes $\mathcal{M}$ and construct a global set of point correspondences $\mathcal{C} = \{(\mathbf{p}_k, \mathbf{q}_k)\}$.

E. Training

1) Loss Function: We train the network as follows:

$$\mathcal{L} = \mathcal{L}_{\text{gnn}} + \mathcal{L}_{\text{shape}}. \tag{14}$$

$\mathcal{L}_{\text{gnn}}$ is proposed to supervise the triplet-GNN and the node match layers, while $\mathcal{L}_{\text{shape}}$ is proposed to supervise the shape backbone. Specifically

$$\mathcal{L}_{\text{gnn}} = \frac{1}{2} \sum_{l=1,2} \sum_{(i,j)\in\bar{\mathcal{M}}} \log \left({}^l\mathbf{A}^X_{i,j}\right) \tag{15}$$

where $\bar{\mathcal{M}}$ is ground-truth node matches and l is the layer index of the node features. Since the dual normalization in (11) already suppresses the negative pairs, $\mathcal{L}_{\text{gnn}}$ has implicitly involved a negative loss term. Thus, we omit the loss penalty regarding the negative match pairs.

The shape loss is constructed by designing a contrastive loss [67] on shape features and an optimal transport loss [41] on the decoded point features $\{\hat{\mathbf{f}}^A_i, \hat{\mathbf{f}}^B_j\}$

$$\mathcal{L}_{\text{shape}} = \frac{1}{2|\bar{\mathcal{M}}|} \sum_{(i,j)\in\bar{\mathcal{M}}} \big(\mathcal{L}_{ot}(\hat{\mathbf{z}}^A_i, \hat{\mathbf{z}}^B_j) + \mathcal{L}_{\text{cont}}(\mathbf{f}^A_j, \mathbf{f}^B_i) + \mathcal{L}_{\text{cont}}(\mathbf{f}^A_i, \mathbf{f}^B_j)\big) \tag{16}$$

where $\bar{\mathcal{M}}$ is the ground-truth node pairs. We explain the contrast loss and optimal transport loss terms as follows:

$$\mathcal{L}_{\text{cont}}(\mathbf{f}^A_i, \mathbf{f}^B_j) = \frac{\exp(\mathbf{f}^A_i \cdot \mathbf{f}^B_j)}{\sum_{k\in\mathcal{N}_i} \exp(\mathbf{f}^A_i \cdot \mathbf{f}^B_k)}$$

$$\mathcal{L}_{ot}(\hat{\mathbf{z}}^A_i, \hat{\mathbf{z}}^B_j) = -\sum_{(u,v)\in\bar{\mathcal{C}}_{i,j}} \log \hat{\mathbf{A}}^Z_{i,j}(u,v) - \sum_{u\in\mathcal{N}^Z_i} \log \hat{\mathbf{A}}^Z_{i,j}(u, K_p) - \sum_{v\in\mathcal{N}^Z_j} \log \hat{\mathbf{A}}^Z_{i,j}(K_p, v)$$

where $\mathcal{N}_i$ is the set of negative nodes compared with $\mathbf{v}_i$, $\bar{\mathcal{C}}_{i,j}$ is a set of the ground-truth point matches between $\{\mathbf{v}_i, \mathbf{v}_j\}$, and $\mathcal{N}^Z_i$ is the set of unmatched points in $\mathcal{P}_i$. The loss term $\mathcal{L}_{ot}$ is designed to supervise the point matching between two associated semantic nodes. It pulls the features of the matched points closer, while pushing the features of the unmatched points further apart.

2) Generate Ground-Truth Match: The ground-truth node match set $\bar{\mathcal{M}}$ and each set of negative nodes $\mathcal{N}_i$ are selected by calculating the intersection between node pairs. We compute the IoU between the nodes' point clouds to decide an intersection. Ground-truth point matches $\mathcal{N}^Z_i$ are those points with their relative distance less than a threshold (i.e., 0.05 m).

V. ROBUST POSE ESTIMATOR

Given a set of correspondences $\mathcal{C} = \{(\mathbf{p}_k, \mathbf{q}_k)\}$, we follow the G3Reg framework [28], which employs a distrust-and-verify approach, for robust transformation estimation. G3Reg generates multiple transformation hypotheses and uses a geometric verification function to select the optimal one.

In the hypothesis proposal step, a pyramid compatibility graph is constructed. Specifically, for two pairs of correspondences $(\mathbf{p}_i, \mathbf{q}_i)$ and $(\mathbf{p}_j, \mathbf{q}_j)$, we test their compatibility

$$\left| \|\mathbf{p}_i - \mathbf{p}_j\|_2 - \|\mathbf{q}_i - \mathbf{q}_j\|_2 \right| < \delta_{ij}. \tag{17}$$

Here, δ_{ij} is a threshold that helps reject potential outliers in the correspondence set $\mathcal{C}$.

By gradually increasing δ_{ij}, we obtain a pyramid graph that becomes denser at each level. For every level, an MAC is identified using the graduated MAC solver from G3Reg [28]. The correspondences in this clique form a potential inlier set $\mathcal{C}^*$. A candidate transformation $(\mathbf{R}^*, \mathbf{t}^*)$ is then determined as

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

follows:

$$\mathbf{R}^*, \mathbf{t}^* = \underset{\mathbf{R}\in \mathrm{SO}(3), \mathbf{t}\in\mathbb{R}^3}{\arg\min} \sum_{(\mathbf{p}_k, \mathbf{q}_k)\in\mathcal{C}^*} \min\left(r\left(\mathbf{q}_k, \mathbf{p}_k\right), \bar{c}^2\right) \tag{18a}$$

$$d_k = \mathbf{q}_k - (\mathbf{R}\mathbf{p}_k + \mathbf{t}) \tag{18b}$$

$$r\left(\mathbf{q}_k, \mathbf{p}_k\right) = \begin{cases} d_k^{\mathrm{T}} \left(\Sigma_y^k + \mathbf{R}\Sigma_x^k \mathbf{R}^{\mathrm{T}}\right)^{-1} d_k, & \text{if instance centers} \\ d_k^{\mathrm{T}} d_k, & \text{otherwise} \end{cases} \tag{18c}$$

where $\bar{c}$ is the inlier cost threshold, and Σ is the statistical covariance matrix of the instance point cloud. This optimization problem can be solved using GNC [68].

We then introduce an evaluation function g to determine the most suitable transformation based on the geometric information of the original point clouds $\mathcal{X}$ and $\mathcal{Y}$

$$\tilde{\mathbf{R}}, \tilde{\mathbf{t}} = \underset{\tilde{\mathbf{R}}\in\{\mathbf{R}_m^*\}, \tilde{\mathbf{t}}\in\{\mathbf{t}_m^*\}}{\arg\min} \; g\left(\mathbf{R}_m^*, \mathbf{t}_m^* \mid \mathcal{X}, \mathcal{Y}\right) \tag{19}$$

where m is the index of a candidate transformation.

The function g is designed based on the geometric distribution of the voxelized point cloud, utilizing point-to-plane and point-to-point distances. More details are available in [28]. The predicted transformation $\mathbf{T}_A^B$ is created from the optimal candidate $(\tilde{\mathbf{R}}, \tilde{t})$.

While the aforementioned framework demonstrates impressive performance for registration with very low inlier ratios (IRs), its efficiency and robustness may be compromised when dealing with higher IRs and repetitive correspondence patterns. To address these limitations, we introduce two practical strategies aiming at enhancing both the efficiency and the robustness of the system.

First, while MAC pruning [69] demonstrates superior robustness at low IRs, its computational cost grows linearly with IR in practice. In contrast, GNC provides comparable accuracy but faster execution at high IRs. Our hybrid approach, therefore, activates MAC only when the GNC-estimated IR falls below a certain value, such as 0.3. Otherwise, we take the prediction from GNC as the candidate transformation. The strategy achieves optimal speed–accuracy tradeoff. Second, repetitive structure tends to create dense and intraconsistent point correspondences, which may construct a large outlier clique and disable the MAC inlier selection strategy [38]. To mitigate this, we apply nonmaximum suppression to the correspondences. For correspondences $(\mathbf{p}_i, \mathbf{q}_i)$ and $(\mathbf{p}_j, \mathbf{q}_j)$, if $\|\mathbf{p}_i - \mathbf{p}_j\|_2^2$ is less than a predefined threshold, we retain only the correspondence with the higher score.

The outlier pruning and the robust pose estimator do not suffer from the generalization issues. They guarantee the registration performance in cross-domain evaluation, which may hold a high outlier ratio.

VI. Two-Agent SLAM

To demonstrate how scene graph registration helps SLAM tasks, we integrate SG-Reg into a two-agent SLAM system to detect loop closures and register the scene graphs. The structure of the implemented system is illustrated in Fig. 5.

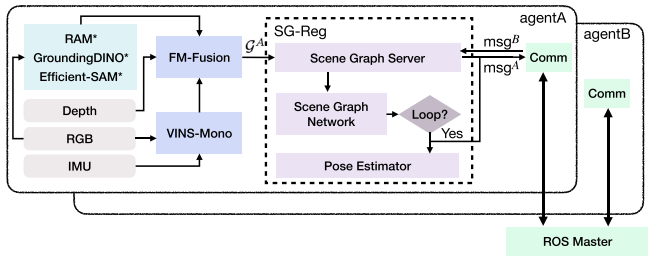


Fig. 5. Two-agent SLAM system structure. Module marked with * runs in offline.

A. Coarse-to-Fine Communication

We introduce a coarse-to-fine communication strategy to achieve optimal performance that balances accuracy and communication bandwidth. Initially, each agent broadcasts its *coarse message* at a steady rate (i.e., 1 Hz), allowing SG-Reg to match semantic nodes. At a query frame, if a minimum number of nodes are matched, the agent sends a request message to its remote agent. In response, the remote agent publishes a *dense message*, enabling the agent to execute the complete registration pipeline shown in Fig. 2. To prevent the dense messages from being published too frequently, we establish a minimum interval between the query frames that allow for sending request messages. In the following, we highlight the format of the broadcast messages from agent-A:

1) coarse message: $\{\mathbf{X}^A, \mathbf{O}^A\}$;
2) dense message: $\{\mathbf{X}^A, \mathbf{O}^A, \mathcal{X}\}$;

where node features $\mathbf{X}^A \in |\mathcal{A}| \times d$, node center points $\mathbf{O}^A \in |\mathcal{A}| \times 3$, and the stacked point cloud $\mathcal{X}$ is as illustrated in Section IV-C.

To enhance the performance of receiving a coarse messages, we maintain the latest point correspondences $\tilde{\mathcal{C}}$ in the program. Upon receiving a coarse message, we merge the matched nodes' center with $\tilde{\mathcal{C}}$ and construct the final correspondence set $\mathcal{C}$, if $\tilde{\mathcal{C}}$ is available. Or we construct $\mathcal{C}$ only using the matched nodes' center. Each time reading a $\mathcal{C}$, SG-Reg predicts a global transformation $\mathbf{T}_A^B$ without an initial transformation.

B. SG-Reg Deployment

We implement the network modules in SG-Reg using LibTorch,[1] enabling us to call the neural network from a C++ executable program. We integrate SG-Reg with other SLAM modules. Each network block in SG-Reg is deployed separately, allowing the SLAM system to match semantic nodes with or without fusing shape features.

C. SLAM Integration

As shown in Fig. 5, we use VINS-Mono [31] to compute visual-inertial odometry. RAM-Ground-SAM [18], [19], [70]

[1] [Online]. Available: https://pytorch.org/cppdocs/

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

TABLE III
DATASET SPLITS FOR TRAINING AND EVALUATION

	Train using 3RScan-GT	Train using ScanNet-Mapping
3RScan-GT evaluation	Sec. VII-B	-
ScanNet-Mapping evaluation	-	Sec. VII-D
3RScan-Mapping evaluation	-	Sec. VII-C

TABLE IV
TRAIN AND EVALUATION ON 3RSCAN-GT

	NR(%)	NP(%)	IR(%)	RR(%)
SG-PGM	63.5	45	15.7	79.4
Ours	**85.2**	**58.1**	**22.5**	**82.0**

processes the RGB sequences and save their prediction before the experiment. FM-Fusion [11] reads the prepared data from RAM-Ground-SAM and runs in real time.

VII. EXPERIMENT

We execute a series of experiments to assess our method. These experiments reveal the following advantages of our approach, which align with our declared novelties in Section I.

1) *Scene graph matching:* Improvement is achieved through the triplet-boosted GNN and shape integration discussed in Section VII-D.
2) *Generalization:* Our method's registration capability is validated on a cross-domain dataset (see Section VII-C) and within a real-world SLAM system (see Section VII-E).
3) *Efficiency:* Encoding the sparse scene representation requires far fewer GPU resources in Section VII-C, enhances communication efficiency in Section VII-E, and accelerates inference speed in Section VII-F.
4) *Self-supervised training:* Our training approach, explained in Section VII-A4, involves data automatically generated from posed RGB-D sequences.

A. Setup

1) Benchmarks: We run our method on two public datasets: ScanNet [71] and 3RScan [29].

The 3RScan dataset provides ground-truth scene graph annotations [16], [17], which we refer to as 3RScan-GT. In addition, considering the difference between ground-truth annotations and real-world data, we use the semantic mapping [11] method to automatically label the two datasets and refer to them as 3RScan-Mapping and ScanNet-Mapping. The division of the training and testing sets follows the original settings of the datasets. As shown in Table III, we train the baseline method and our method using 3RScan-GT and ScanNet-Mapping separately. We evaluate their cross-domain performance in Section VII-C.

Beyond reconstructed scenes, we evaluate our method in a customized two-agent SLAM benchmark. Our scene graph registration modules are deployed into a SLAM system, and it registers scene graphs between two agents in a coarse-to-fine strategy. The benchmark is based on RGB-D and inertial sequence data that are collected in the real-world environment.

2) Evaluation Metrics: Node recall (NR) and node precision (NP) evaluate scene graph matching performance. If the IoU between the nodes' point cloud is higher than a threshold $\tau_{\text{iou}} = 0.3$, we treat it as a true positive (TP) node pair. Otherwise, it is a false positive (FP) pair. Then, we can compute the NR and NP, similar to the recall and precision in image matching task [6]. In the registration task, we follow GeoTransformer [41] and use IR and RR to evaluate the accuracy. With the predicted transformation, if the root-mean-square error (RMSE) between the aligned point cloud is within a threshold (i.e., RMSE$<$ 0.2 m), we treat the prediction as a successful registration. RR is the portion of the successful registration.

3) Baselines: We compare our registration performance against SG-PGM [17] and GeoTransformer [41]. In two-agent SLAM, our approach is compared with Hydra [15], [50] and HLoc [30]. Hydra and HLoc executes offline in the benchmark.

4) Training Data: In ScanNet-Mapping data, each ScanNet sequence is segmented into multiple sub-sequences. We select a pair of the subsequences as a source and a reference sequence. A random transformation in four DoFs is incorporated into each pair of the scene graphs. We run FM-Fusion on each subsequence. Hence, we can have a large number of scene graph pairs for training and evaluation. Since the ground-truth annotation from ScanNet is not used, we claim a *self-supervised scene graph training* is used in our method. The training method can be extended to other indoor RGB-D SLAM data, and it does not require the ground-truth semantic annotation. In 3RScan-GT data, we directly applied the data generation from SGAligner [16].

5) Implementation: We train our network using an Adam optimizer with a learning rate of 0.01. The training data involve 1990 pairs of scene graphs reconstructed from ScanNet-Mapping and the 3RScan-GT dataset. The pretrain of the shape network takes 64 epochs. The rest of the training takes 80 epochs. We run the training process and all experiments on a desktop computer with an Intel-i7 CPU and a Nvidia RTX-3090 GPU.

B. 3RScan Benchmark

This section evaluates the differences in accuracy between different methods using completely accurate training and validating annotations. We use the officially published version of SG-PGM [17] and train and validate our model and SG-PGM using 2178 pairs of scene graphs from 3RScan-GT. We make two adaptions in our trained SG-PGM. First, we use BERT to encode their semantic labels, which is identical to our method. The original version of SG-PGM generates a semantic histogram from each of its semantic labels and constructs a semantic feature vector. Since Bert is a stronger semantic encoder, we apply it in SG-PGM to ensure a fair comparison. Second, SG-PGM reads the relationship labels of the edges. We assign all of the relationship labels to *none*. This is because the scene graphs in 3RScan-Mapping do not provide any relationship labels, and we set them to none in the two versions of the training.

As shown in Table IV, our NR and NP are significantly higher than SG-PGM. Our RR is slightly higher than that of SG-PGM. This reflects that we maintain a decisive advantage in semantic

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SCENE POINTS NUMBER IN EACH SPLIT

	Small	Median	Large
Points in thousand(k)	61k to 133k	146k to 210k	210k to 330k

All the scene points are downsampled at 2-cm voxel size.

node matching. However, our advantage in RR is less significant. This is probably because using ground-truth semantic annotations will improve the accuracy of node matching. So, our method cannot fully demonstrate its robustness and superiority.

For scene graph registration, real-world data always contain segmentation variances and false predicted semantic labels, which greatly affect accuracy. Therefore, we believe training on 3RScan-GT cannot reflect the actual usability of the scene graph registration method but shows an upper bound in accuracy. The experimental results of this section are only used as a reference to prove that our method is also superior to the baseline method in an ideal application scenario.

C. Cross-Domain Benchmark

Deep learning methods are often limited by their generalization performance. Considering that our method aims to be applied in the long-term use of robots, we believe it is necessary to evaluate the cross-dataset performance. In this section, we evaluate in detail the generalization, accuracy, and efficiency of our method and two baselines on a validation dataset that is different from their training dataset.

1) Baseline Setup: We use the officially published version of GeoTransformer [41] and SG-PGM [17]. We train them using ScanNet-Mapping and evaluate them in 3RScan-Mapping to test the generalization ability in the different data domains. We make similar modifications to the realization of SG-PGM, as in Section VII-B. We keep all the parameters in the 3-D backbone identical, such as voxel size and point feature dimension. We also turn OFF the ICP [72] refinement for all the methods to ensure a fair comparison.

2) Registration Accuracy: We split the 100 pairs of scenes from 3RScan-Mapping into three sets: 76 small pairs, 13 medium pairs, and 11 large pairs. The grouping is based on the maximum number of points in each pair. We summarize the scene points range of each split, as shown in Table V.

As the median-size scene shown in Fig. 6, the semantic scene graphs in 3RScan-Mapping exhibit significant semantic noise, including oversegmented objects and noisy point cloud reconstruction. Our method generates more correct node matches and fewer false node matches than SG-PGM. It demonstrates preciser and more robust scene graph matching performance than SG-PGM. Compared with GeoTransformer, our semantic nodes are much sparser than the superpoints from GeoTransformer. Thanks to the strong descriptiveness features, our node matching precision is higher than the superpoint matching precision in GeoTransformer.

To discuss the registration performance, we introduce the definition of *pseudo inliers*, the correspondences after outlier pruning and being treated as inliers by the estimator. As introduced in Section V, we prune the outliers in point correspondences via MAC. Similarly, GeoTransformer and SG-PGM prune outliers via a local-to-global registration [41], based on a weighted SVD [44]. The baseline works and SG-Reg all have pseudo inliers. We compute the pseudo inlier ratio (PIR), the ratio of true inliers within the pseudo inliers. As shown in Fig. 6, our IR and PIR are higher than those of GeoTransformer, demonstrating more accurate point matching. In the final registration result, both SG-Reg and GeoTransformer successfully align the scenes.

However, in SG-PGM, the presence of adversarial outliers leads to registration failure. As shown in Fig. 6, these outliers are geometrically consistent and thus more challenging to remove. Despite efforts to prune outliers, these adversarial outliers persist in the pseudo inliers. While adversarial outliers also appear in GeoTransformer and our result, SG-PGM is particularly susceptible as it cannot effectively prune them due to its high outlier ratio. In addition, SG-PGM's outlier pruning method, which relies on a batch of weighted SVD [44], may impede its ability to eliminate adversarial outliers. It searches for a mini-batch of correspondences that contains the largest number of pseudo inliers. Consequently, this mini-batch, which includes the adversarial outliers, is likely to receive the highest confidence score, leading to a false SVD prediction.

TABLE VI
SCENE GRAPH REGISTRATION PERFORMANCE

		NR(%)	NP(%)	IR(%)	RR(%)
Small	GeoTransformer	47.9	**47.9**	14.5	76.3
	SG-PGM	35.2	22.5	5.5	40.8
	Ours	**66.8**	40.6	**20.4**	**80.3**
Median	GeoTransformer	25.6	27.1	10.6	61.5
	SG-PGM	26.8	15.7	3.9	7.7
	Ours	**58.8**	**36.9**	**19.9**	**76.9**
Large	GeoTransformer	-	-	-	-
	SG-PGM	26.2	12.0	1.6	27.3
	Ours	**51.9**	**32.8**	**14.4**	**72.7**
Overall	GeoTransformer	43.8	**41.9**	13.9	74.2
	SG-PGM	32.9	20.0	5.0	35.0
	Ours	**64.9**	38.9	**19.5**	**79.0**

In each split, the **best** metrics are presented in bold and the second best metric are underlined.

Then, we summarize the quantitative results in Table VI. As shown in Table VI, our overall performance is much better than SG-PGM. We outperform SG-PGM in each level of matches and the final registration results. Considering that our method and SG-PGM have adopted a strategy of explicitly fusing node features and shape features, this significant accuracy gap further demonstrates the scientific validity of our encoding method. It also validates that our method is able to register scene graphs across data distribution, demonstrating strong generalization ability.

Compared with GeoTransformer, our method achieves slightly better performance, improving RR by 4.8% and the IR by 5.6%. However, in median-sized scenes, we significantly outperform GeoTransformer, achieving a 15.4% higher RR. This demonstrates that the primary advantage of incorporating semantic information for registration lies in achieving higher recall in larger scenes. In larger scenes, frequent similar geometric structures exist, such as point clouds from multiple chairs

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

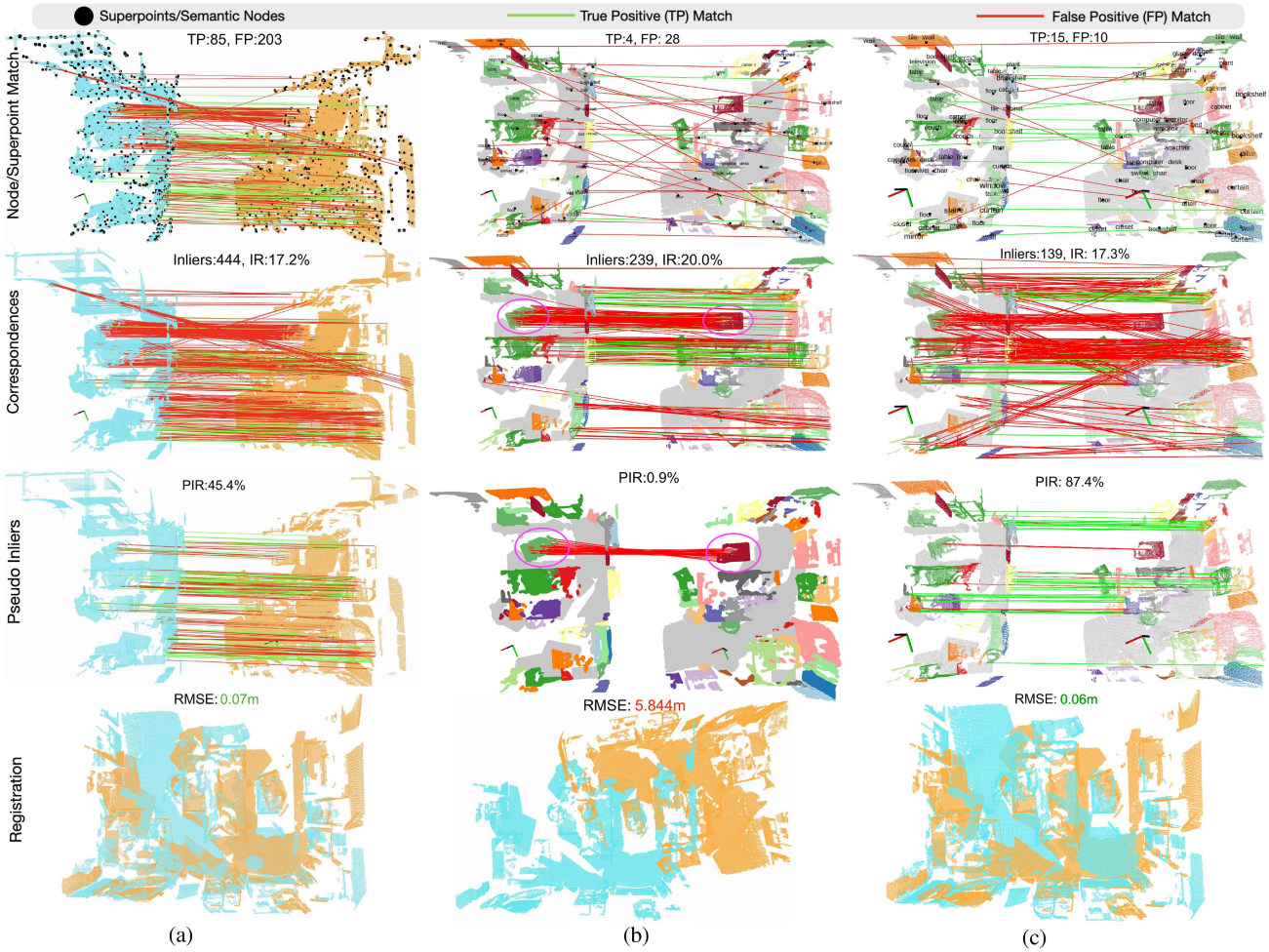


Fig. 6. Registration results in a 3RScan median scene. In SG-PGM results, the adversarial outliers are highlighted in purple circles. We annotate the corresponding evaluation metrics in each subfigure. The RMSE from a successful registration is highlighted in green, while the RMSE from a failed pair is in red. (a) GeoTransformer. (b) SG-PGM. (c) SG-Reg (Ours).

TABLE VII
ANALYSIS POINT CORRESPONDENCES AND THEIR PSEUDO INLIERS

	GeoTransformer	SG-PGM	**Ours**
Inlier Rate (%)	14.2	5.0	**19.5**
Pseudo Inlier Rate (%)	31.5(+17.3)	18.7(+13.7)	**50.3**(+30.8)

The improvements after prune outliers are highlight as a precision gain.

or numerous books. Relying solely on 3-D point features to register large scenes is extremely challenging. By incorporating semantic information, we enhance the descriptive ability of the encoded 3-D features.

We also report the pseudo inliers to support our improvement. As shown in Table VII, our PIR is higher than that of SG-PGM and GeoTransformer. A higher PIR increases the likelihood that the pose estimator will predict an accurate transformation $\mathbf{T}_A^B$. In addition, we note that each method in Table VII has a PIR greater than its IR. However, our PIR has improved by 30.8%, which represents a larger gain than the baseline. This suggests that SG-Reg generates fewer adversarial outliers, allowing for more effective pruning of outliers and resulting in a greater precision gain after outlier removal.

3) Computational Efficiency: SG-Reg requires significantly fewer floating-point operations per second (FLOPS) and consumes less GPU memory compared to baseline models. This enhanced computational efficiency arises from its ability to encode scenes at a higher compression ratio. As illustrated in Fig. 7(a), SG-Reg reduces a scene to dozens of semantic nodes, while GeoTransformer compresses it into hundreds of superpoints. Given that attentional operations [46] on an input feature set have a computational complexity of $O(n^2)$, where n is the number of input features, performing attentional operations on semantic nodes is far more computationally efficient than on superpoints.

To verify our analysis, we design a study that runs a single GNN layer from GeoTransformer, SG-PGM, and SG-Reg. We conduct the study in all of target scenes. As shown in Fig. 7(b), SG-Reg consumes the least GPU memory and requires far fewer FLOPS than the two baseline models. Even for large scenes exceeding 210K points, our reserved GPU memory remains below 11 GB, and the FLOPS are below 2215 MFLOPS. In contrast, GeoTransformer requires more than 1 000 000 MFLOPS, significantly higher than ours. Besides, GeoTransformer cannot inference the large scenes due to memory limitations; it exceeds the maximum GPU memory

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

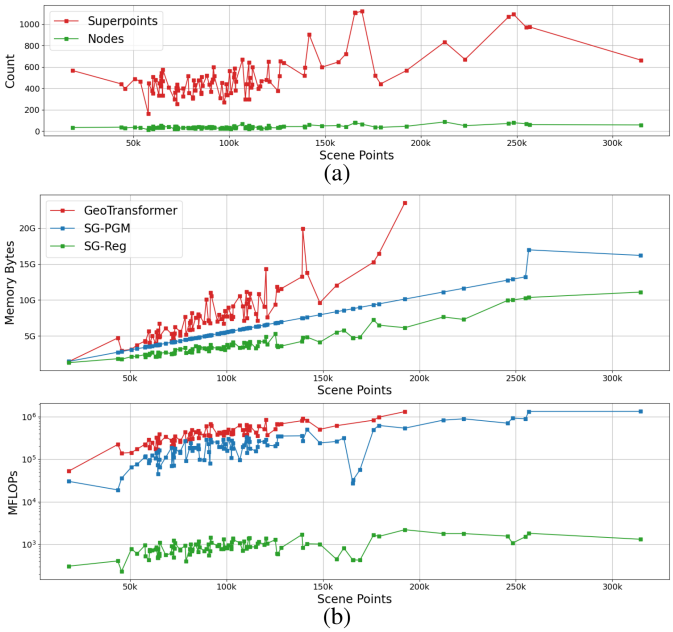


Fig. 7. (a) Sparsed 3-D representations from the 3RScan scenes in the cross-domain benchmark. (b) Reserved memory and computational complexity on a Nvidia RTX-3090 GPU. The MFLOPS are shown in exponential scales.

TABLE VIII
EVALUATE REGISTRATION IN SCANNET-MAPPING

	NR(%)	NP(%)	IR(%)	RR(%)
SG-PGM	64.6	53.3	29.1	94.0
Ours	**86.2**	**63.1**	**63.4**	**99.1**

capacity of 24 GB on the Nvidia RTX-3090. SG-PGM, our closest baseline, exhibits efficiency that falls between SG-Reg and GeoTransformer, as it inherits the superpoint encoding from GeoTransformer, necessitating more GPU resources than our approach.

D. ScanNet Benchmark

We further evaluate our performance in ScanNet-Mapping evaluation split, which includes 218 pairs of scene graphs. In the ScanNet benchmark, apart from overall accuracy, we focus on exploring the factors that influence scene graph matching, including the effects from GNN backbone, shape fusion strategy, and perception field of the semantic nodes.

1) Registration Comparison: As shown in Table VIII, without considering cross-domain performance, our NR and NP are largely higher than SG-PGM. Our RR is slightly higher than SG-PGM. We report the source domain results to show each method's upper bound.

2) Triplet-Boosted GNN Evaluation: We demonstrate that the triplet-boosted GNN can learn the node features with stronger spatial awareness. Three baseline GNN backbones are used as baselines: vanilla GAT [25], the RPE GAT used in LightGlue [7], and Geometric Transformer [41]. We apply each GNN backbones to replace the triplet-boosted GNN we use. All methods are trained and evaluated on ScanNet. To focus on evaluating the influence of GNN, we skip the shape fusion module in all the baseline methods and our method. We directly use the node features after the GNN to match the semantic nodes.

TABLE IX
EVALUATION OF GNN LAYER ON THE SCANNET-MAPPING SPLIT

	Node Rec.(%)	Node Pre.(%)
Vanilla GAT	72.9	47.7
RPE GAT	75	52.7
Geometric Transformer	71.1	46.2
Triplet-boosted GNN (**Ours**)	**76.8**	**57.5**

To focus on evaluating the GNN layers, the shape features are not fused in this study.

In the office scene depicted in Fig. 8, the triplet-boosted GNN generates more TP matches and fewer FP matches. The scene features multiple objects within the same semantic categories, positioned closely together. The vanilla GAT only considers a semantic topological relationship. It struggles with these ambiguous objects, resulting in the highest number of FP matches. In contrast, the RPE GAT, which is boosted by a variant relative position information, performs slightly better than the vanilla GAT. Furthermore, since the triplet feature is invariant to a global transformation in four DoFs, the triplet-boosted GNN outperforms the RPE GAT.

Next, we analyze the quantitative results. As shown in Table IX, our method achieves the highest NR and NP in scene graph matching. It proves that the triplet-boosted GNN can learn the local topological relationships while distinguishing their spatial distribution. On the other hand, the other GNN backbones perform less satisfactorily. Interestingly, we find that the geometric transformer performs even poorer than a vanilla GAT. In point cloud registration tasks, geometric transformer [41] is used to encode superpoints and perform very well. However, in scene graph registration tasks, semantic nodes have a larger geometric variance. Many of them are partially observed, with some being oversegmented. This may introduce extra noise to a node center, which is later propagated into its learned geometric embedding. As a result, geometric transformer performance is degenerated.

3) Evaluate Shape Fusion: We run an ablation study to evaluate the impact of shape feature. In the no fusion setting, the node features $\{{}^1\mathbf{X}^A, {}^1\mathbf{X}^B\}$ after the triplet-boosted GNN are used to search node matches. In the early fusion setting, shape features are fused into node features $[{}^0\mathbf{X}^{A/B} \| \mathbf{F}^{A/B}]$ before the triplet-boosted GNN, while in late fusion, they are fused into node features $[{}^1\mathbf{X}^{A/B} \| \mathbf{F}^{A/B}]$ after the GNN.

We visualize the scene graph matching results using different fusion approaches in Fig. 9. As shown, a few shelves and a table are incorrectly matched in the no shape fusion mode but are rejected after shape fusion due to their differing shapes. In addition, some objects are correctly matched after shape fusion, which were missed in the no fusion mode due to semantic noise. This demonstrates how the fused shape features enhance the performance of scene graph matching.

We summarize the quantitative results of the fusion approaches in Table X. As shown, the late fusion strategy outperforms the others in graph matching. This suggests that while the shape feature is less critical for learning the graph topology,

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

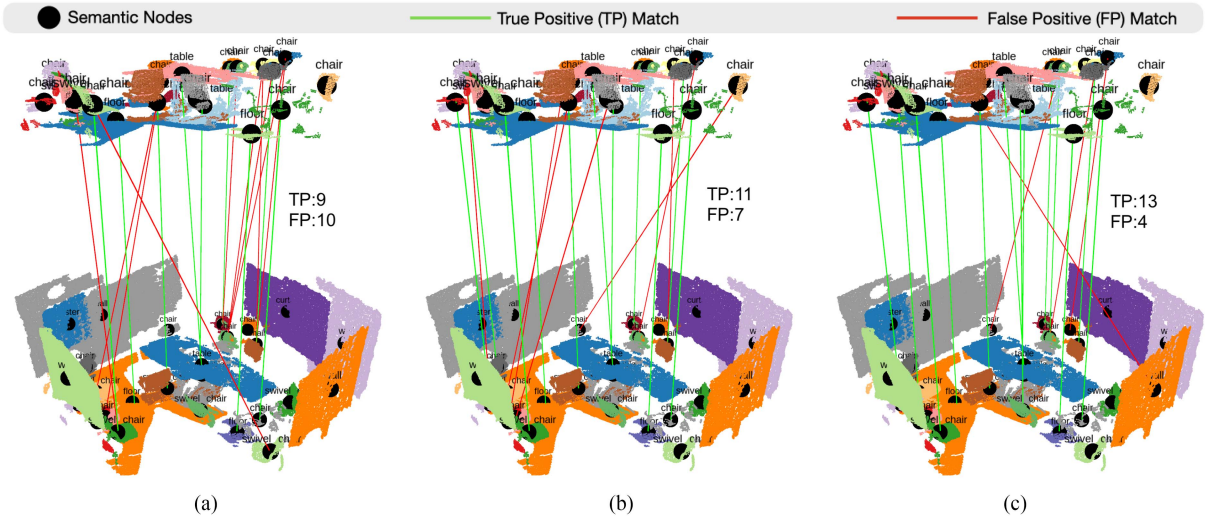


Fig. 8. Visualize the scene graph match result from different GNN backbone. Results are from ScanNet *scene0655_02*. (a) Vanilla GAT. (b) RPE GAT. (b) Triplet-boosted GNN (Ours).

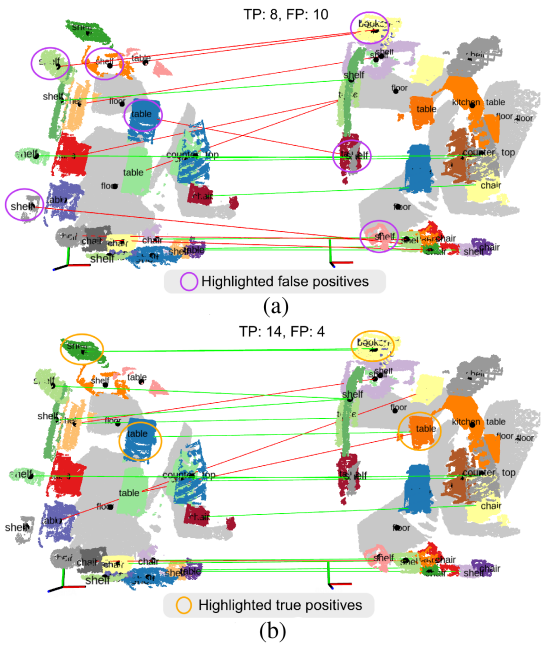


Fig. 9. Scene graph matching at a ScanNet scene. We highlighted a few FPs that are rejected after the fusion, and a few TPs that are matched after the fusion. (a) No shape fusion. (b) Late fusion.

TABLE X
ABLATION STUDY ON THE FUSION STRATEGY

	Node Rec.(%)	Node Pre.(%)
No fusion	76.8	57.5
Early Fusion	79.8(+3.0)	60.7(+3.2)
Late Fusion (**Ours**)	**86.2**(+9.4)	**63.1**(5.6)

The improvements relative to the no fusion mode are annotated.

it becomes more significant when fused after the triplet-boosted GNN. In this work, we employ the late fusion strategy, as illustrated in our system structure in Fig. 2.

E. Two-Agent SLAM Benchmark

The following experiment estimates the relative localization between agent-A and agent-B. The agents have their local coordinates system, denoted as O^A and O^B. At a query frame, whose frame index is q, the agent pose estimation in its local coordinate is $\mathbf{T}_q^A$ or $\mathbf{T}_q^B$. The benchmark focuses on registering the local coordinates between two agents and predicting a relative transformation $\mathbf{T}_A^B$, where $\mathbf{T}_A^B \in SE(3)$.

1) Data Collection: We collect the experiment data using a visual-inertial hardware suite [73]. It has an Intel RealSense D-515 camera to capture RGB-D sequence and a DJI A3 flight controller that provides IMU data streaming. The experiment involves ten pairs of RGB-D sequences inside an academic building. Among them, five pairs of sequences have camera trajectories in opposite directions, as the example scene shown in Fig. 1. We generate ground-truth transformation $\bar{\mathbf{T}}_A^B$ by manually aligning the reconstructed point cloud, following with an ICP refinement.

2) Evaluation Metrics: With the predicted transformation $\mathbf{T}_A^B$ and ground-truth transformation $\bar{\mathbf{T}}_A^B$, we compute their *relative translation error* (RTE) and the *relative rotation error* (RRE). If RTE$<$ 0.2 m and RRE $< 5°$, we call it a successful registration.

3) Using Hydra and HLoc: When using Hydra and HLoc in agent-A, we assume that it has received all the keyframes broadcasted from agent-B between time stamp $[t_0, t_q]$, where t_q is the time stamp at the query frame. The image keyframes in agent-A and agent-B are $\{\mathcal{I}_i^A\}_i$ and $\{\mathcal{I}_i^B\}_i$, where the frame index $i \in [0, q]$.

In the Hydra implementation, we mainly utilize its loop closure detection module.[2] We input our scene graph into Hydra, which is used to construct the handcrafted scene graph descriptor, as introduced in [15] and [50]. The scene graph sent to Hydra is identical to the one sent to our method. Hydra

[2][Online]. Available: https://github.com/MIT-SPARK/Hydra

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

retrieves matched frames using the handcrafted descriptor during the global matching step, identifying the top-ten images from the set $\{\mathcal{I}_i^B\}_i$. It then verifies the matched images using DBoW2. In the local matching step, we implement the image registration module from Kimera.[3] Specifically, it performs brute-force matching of ORB features between the query image $\mathcal{I}_q^A$ and Hydra's candidate images. The candidate image with the highest number of matched ORB features is selected as the final matched image $\mathcal{I}_l^B$. Subsequently, it uses the query depth image $\mathcal{D}_q^A$ and the image correspondences to solve a RANSAC-based PnP problem [33], predicting a relative transformation $\mathbf{T}_q^l$. By utilizing their local pose estimation $\mathbf{T}_q^A$ and $\mathbf{T}_l^B$, the implemented Hydra predicts a $\mathbf{T}_A^B$.

We noticed that the latest version of Hydra [50] introduces a GNN-based scene graph descriptor. However, it does not publish the source code or its model weight. So, we can only choose the handcrafted descriptor to compare.

In the implemented HLoc,[4] it runs NetVLAD to retrieve the top-k matched images and we set $k = 10$. Next, it runs LightGlue to match features in each of the retrieval images. HLoc selects the candidate image with the most matched features as the final loop image $\mathcal{I}_l^B$. Similar to the pose estimation in Hydra, we take the image matching result and the depth image $\mathcal{D}_q^A$ to run RANSAC-based PnP [33]. With their local pose estimation, we predict the relative transformation $\mathbf{T}_A^B$.

We run Hydra and HLoc in an offline setting. They can access the same data streaming from the two agents as our method. In designing the communication strategy for HLoc and Hydra, we follow the communication mechanism in D^2SLAM [1], which relies on image matching modules to detect loop closure. We assume that each agent incrementally broadcasts its live image and live image features (i.e., NetVLAD and DBoW). And we record the bandwidth that belongs to each type of message.

4) Registration as a Rigid Body: It is important to note that this benchmark focuses on evaluating the registration between agents while neglecting the pose drift within the trajectory of a single agent. In a previous multiagent benchmark [74], the system relies on Hydra to detect loop closures across agents. The loop closure measurements are combined with odometry constraints to construct a pose graph, which is then further optimized. In this study, we evaluate only the registration performance, treating each local trajectory or scene graph as a rigid body representation.

5) Running Rate: We run Hydra, HLoc, and SG-Reg in every ten keyframes. To ensure a fair comparison, we standardize the number of query frames across all methods. Specifically, even if a method identifies too few correspondences, we still execute the registration process to predict a transformation. As a result, the number of query frames for each method remains nearly identical.

6) Robust Pose Average: Since a single estimation at a query frame can be noisy, we run the robust pose average from Kimera-Multi [75] in multiple consecutive query frames. Specifically, the predicted poses at the query frame and the previous X queried frames are collected and sent to the robust pose average function, where X is the selected window size. The pose average module generates a refined pose as the final result. It is designed to enhance the stability of the method. If multiple relative transformations are predicted with a few outliers, pose averaging should improve the performance. On the other hand, if the predictions are robust, the robust pose average module brings limited enhancement. Thus, it can be used to verify the stability of the predicted relative transformations.

7) Qualitative Results: We start the evaluation by visualizing a sample loop closure detection frame. As shown in Fig. 10, Hydra recalls a false loop closure. The semantic histogram at the query frame and the false loop frame are closely similar, resulting in an ambiguous handcrafted descriptor. Their DBoW descriptors are similar as well. So, Hydra matches them in the global retrieval step. Then, their ORB features are also falsely matched. It demonstrates a false loop closure in large viewpoint differences. The images observing different subvolumes create two closely similar handcraft descriptors. Similarly, HLoc fails at the same query frame, as shown in Fig. 10. It falsely matches an unrelated image due to its similar appearance. On the other hand, our method searches correspondences between 3-D scene graphs, which tolerates vastly larger viewpoint differences. Once the scanned 3-D scene graphs overlap, our method can successfully register the scenes.

We further visualize the registration results on a 3-D point cloud map with sequences of camera poses. As shown in Fig. 11, our method registers the scenes at a higher success rate than Hydra and HLoc. Hydra fails most of the query frames due to its handcrafted semantic descriptor and the classical ORB-based image matching. The handcrafted semantic descriptor cannot capture the spatial distribution of the objects. So, it frequently matches the FP images. Moreover, their ORB-based image matching suffers from a significant viewpoint difference. Thus, most of its querying frames fail.

HLoc's success rate is much higher than that of Hydra. However, it still generates around 60% failed registration frames. The two agents move in vastly different trajectories, generating significant different viewpoints. NetVLAD and LightGlue are both affected by the viewpoint differences. So, their success rate is less satisfied.

Another interesting phenomenon is that our query frames with success registration are spatially close to each other, as shown in Fig. 11. This is because our method requires a longer initialization. At the beginning of the sequence, our integrated scene graphs have very few semantic nodes and a low overlap ratio. As the scene graphs grow, they gradually have a larger intersection. Hence, nearly all of our query frames failed initially, while those in the latter half of the sequence were mostly successful. Conversely, as shown in Fig. 11, the successful loop closure frames in HLoc are spatially distributed in variances.

To verify the stability of the three algorithms, we conducted a robust pose-averaging experiment to refine the predictions. According to our previous introduction, the pose averaging rejects outliers in the estimated transformation. Therefore, when the algorithm is robust enough to noise, pose averaging has a

[3][Online]. Available: https://github.com/MIT-SPARK/Kimera-VIO
[4][Online]. Available: https://github.com/cvg/Hierarchical-Localization

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

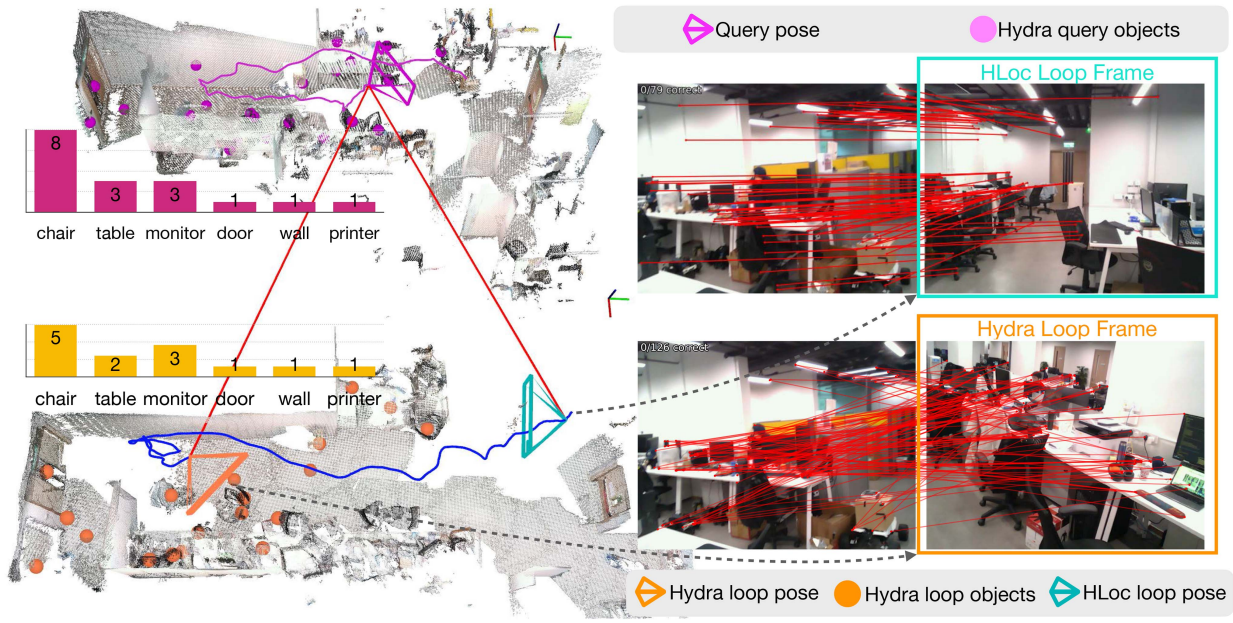


Fig. 10. Loop closure visualization of Hydra and HLoc at a sampled query frame. On the left, 3-D maps from the two agents are rendered, along with the poses of the query frame and the loop frames. In addition, the semantic objects used by Hydra are rendered on the 3-D maps, while their semantic histograms are separately displayed. On the right, the image matching results for the query frame are presented. At the query frame, HLoc achieves RTE at 15.1 m and Hydra achieves RTE at 17.4 m, both fail the registration.

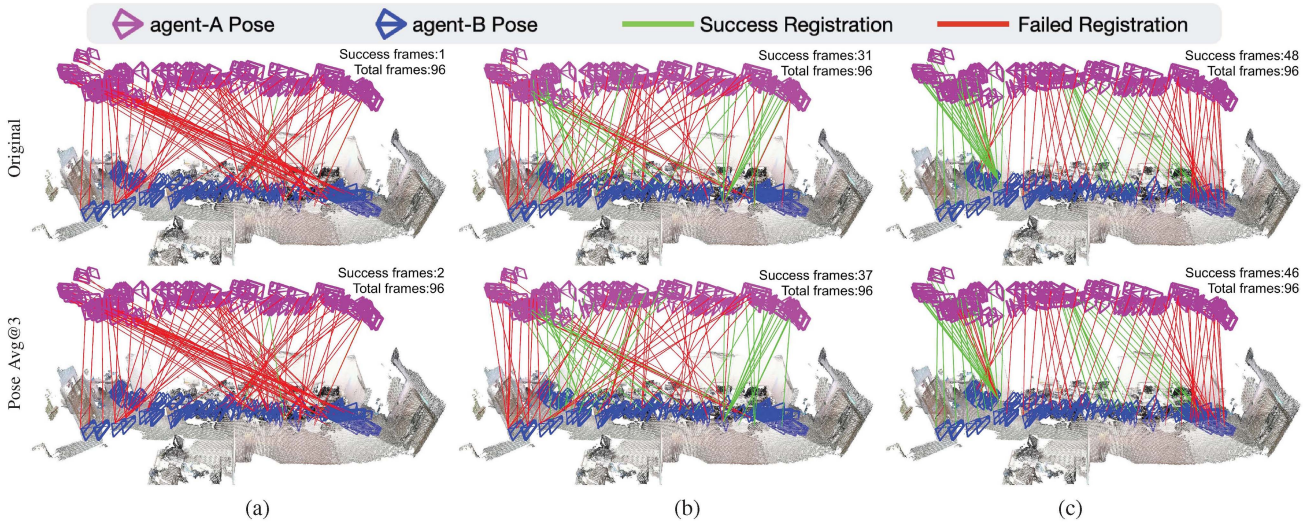


Fig. 11. Registration visualization in an office scenario. The first row shows the original results, while the second row shows the results after pose averaging using three nearest measurements. Each registration result is represented in a line. A registration is successful if its predicted camera pose has an RTE < 0.2 m and RRE $< 5°$. For a better visualization, we align the camera poses with ground-truth transformation and incorporate a vertical translation. (a) Hydra. (b) HLoc. (c) SG-Reg (Ours).

minimal impact. When the algorithm has drastic randomness, its accuracy can be improved after pose averaging.

As shown in Fig. 11, pose averaging has a minimal impact on Hydra due to its low success rate. In contrast, HLoc achieves a higher success rate after pose averaging, demonstrating that HLoc is significantly affected by random noise. This also highlights the effectiveness of the pose averaging module in estimating a reliable pose from multiple predictions with variations. However, as shown in Fig. 11, pose averaging has little effect on our results. We attribute this to the inherent stability of our method: while it requires a relatively higher scene overlap and adopts a conservative approach to registration, it consistently achieves accurate and stable results without significant jitter.

Considering HLoc is a powerful baseline, we explain the main differences between HLoc and our method. Our method first performs object-level registration based on semantic center points and further does point-level refinement on the objects. Therefore, our method utilizes significantly fewer feature points than HLoc and occupies a much smaller transmission bandwidth. However, our method requires more data for initialization. When the scene

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
EVALUATE REGISTRATION SUCCESS RATE (SR)

	Original		Pose Avg@3		Pose Avg@5	
	SR (%)	RTE(m)	SR (%)	RTE(m)	SR (%)	RTE(m)
Hydra	3.7	0.108	4.7	0.111	4.1	0.113
HLoc	31.7	0.118	**34.1**	0.121	**35.6**	0.119
Ours	**32.4**	**0.106**	32.0	**0.105**	31.5	**0.107**

We report the averaged RTE from the succeed registration. Pose Avg@X refers to the robust pose average using X frames.

IoU is too low, our method may be unable to find overlapping objects, leading to registration failures. On the other hand, HLoc uses more feature points and requires higher computational complexity, but it increases the possibility of registration success in low repetition scenarios.

In the following comparison with HLoc, we aim to verify whether our method can achieve similar or even better accuracy than HLoc with much less communication bandwidth and faster inference speed.

8) Quantitative Results: As shown in Table XI, Hydra recalls very few success frames. It is because Hydra focuses more on the back end of scene graph optimization, which involves bundle adjustment [23] and mesh optimization. In this benchmark, we evaluate the registration accuracy at a single loop closure measurement, which makes Hydra less competitive. Specifically, Hydra suffers from limited descriptiveness due to its handcraft semantic descriptor, frequently false retrieval images from DBoW2, and inaccurate image matching by the brute-force method.

HLoc is a competitive baseline with which to compare. Surprisingly, our success rate is higher than that of HLoc before conducting pose averaging. After the pose averaging refinement, as explained earlier, HLoc improves, while our method remains unaffected. Therefore, HLoc shows a better success rate. However, our method always maintains an advantage in accuracy. This means that our method tends to be conservative but precise. HLoc performs registration even when the IoU is small, which may result in matching results with significant bias.

The experiment results also align with our initial estimation of the advantages and disadvantages of HLoc and our method. As shown in Fig. 12, HLoc can provide registration earlier and at a higher registration success rate when the scene overlap is low. However, it is easily affected by noise and can hardly cope with large changes in perspective. Our method requires a long initialization time, but it can ensure the stability of registration accuracy, pose a sustained growth in success rate, and is robust to significant relative viewpoints. Considering that our method uses much less bandwidth, its drawbacks are completely acceptable.

Meanwhile, we point out that SG-Reg is not adversarial with image registration methods. In future works, they can be combined into a unified factor graph to optimize.

9) Communication Bandwidth: A significant advantage of our method is the communication bandwidth. As shown in Table XII, our method only takes 3.9% of the communication bandwidth that HLoc would take at each query frame. Even if we exclude the RGB images from HLoc, we require less than 11% bandwidth of theirs. The enhanced communication efficiency



Fig. 12. Loop closure success rate over the sequence length. Avg@X refers to the robust pose averaging in X consecutive loop frames.

TABLE XII
COMMUNICATION BANDWIDTH COMPARISON

	Message	Number	Dimension	Bandwidth
Hydra	SG-Descriptor	1131	339	1,497 kB
	ORB	622k	32	77,746 kB
	RGB	1,131	640 × 480	1,017,900 kB
	Overall	-	-	970 kB/Frame
HLoc	NetVLAD	1131	4096	18,096 kB
	SuperPoint	460k	259	463,864 kB
	RGB	1,131	640 × 480	1,017,900 kB
	Overall	-	-	1326.1 kB/Frame
Ours	Nodes	17k	132	8,472 kB
	Points	2918k	4	40,347 kB
	Overall	-	-	**52 kB/Frame**

Our messages contain the semantic nodes and points. Their message format is introduced in Section VI. We report the accumulated number of each representation. The corresponding averaged bandwidth is in the overall rows.

TABLE XIII
ABLATION STUDY OF COMMUNICATION STRATEGY

	Averaged interval time(s)		Success rate	BW
	Coarse message	Dense message		
A	0.98 s	None	34.9%	10.1 KB
B	0.98 s	9.5 s	57.8%	109.1 KB
C	0.98 s	5.4 s	62.3%	163.6KB
D	0.98 s	3.4 s	63.3%	223.7 KB

Setting a turns OFF the dense message broadcast and fully relies on the coarse messages. BW refers to the averaged bandwidth over the query frames.

is due to the sparsified scene representations. We broadcast the encoded node features in the coarse message, which requires an extremely low data bandwidth. We publish a coarse message in every registration frame and only publish a dense message if the other agent requests it. It reduces the amount of dense messages that need to be published.

To further evaluate the influence of communication rate and registration accuracy, we run an ablation study. It defines a minimum interval time for publishing dense messages. We change the interval time and run the experiment in four settings. Results from each setting are summarized in Table XIII.

As shown in Table XIII, from setting A to setting D, the success rate gradually increases as the interval time between

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

TABLE XIV
ANALYSIS COMPLEXITY IN COMPUTING THE SIMILARITY MATRIX

	Input Dimensions $\tilde{\mathbf{Z}}^A$	Input Dimensions $\tilde{\mathbf{Z}}^B$	Dot Product Operations
DM	$(\lvert\mathcal{A}\rvert \cdot K_p) \times d_z$	$(\lvert\mathcal{B}\rvert \cdot K_p) \times d_z$	$(\lvert\mathcal{A}\rvert \cdot K_p) \cdot (\lvert\mathcal{B}\rvert \cdot K_p)$
CFM	$\lvert\mathcal{M}\rvert \times K_p \times d_z$	$\lvert\mathcal{M}\rvert \times K_p \times d_z$	$\lvert\mathcal{M}\rvert \cdot K_p \cdot K_p$

DM denotes dense matching, while CFM denotes coarse-to-fine matching.

TABLE XV
COMPARE RUNTIME IN MILLISECOND (MS) ON 3RSCAN BENCHMARK

Steps	GeoTransformer	SG-PGM	**Ours**
Point&Shape Backbone	23.6	60.1	20.3
GNN&Attention Layers	71.2	59.0	2.45
Match Layers	91.7	52.7	28.6
Pose Estimator	15.7	10.9	$90.5^{\dagger}$
Total	202.3	182.7	**142.7**

We mark our pose estimation time using † because it runs on CPU, while the baselines estimate pose on GPU.

publishing dense messages decreases. This demonstrates that dense messages significantly improve the likelihood of successful registration. However, the success rate approaches its optimal level at a certain communication rate, beyond which further improvements are marginal. In this experiment, we selected the setting of Group C for its optimal balance between registration success and communication efficiency.

10) Coarse-to-Fine Versus Dense Matching: To investigate how the coarse scene graph matching improves registration performance, we conducted an ablation study by disabling coarse-to-fine matching and directly applying dense matching. Specifically, the dense matching method brute-force matches point features between the two scenes, while the coarse-to-fine method (ours) matches point features only between matched semantic nodes.

In the *coarse-to-fine point matching*, we select the point features from the matched nodes $\mathcal{M}$ and build $\{\tilde{\mathbf{Z}}^A, \tilde{\mathbf{Z}}^B\}$. We then compute the similarity matrix

$$\mathbf{S}^Z = g(\tilde{\mathbf{Z}}^A, \tilde{\mathbf{Z}}^B) \tag{20}$$

where $g(\cdot)$ is the similarity function in (12). In *dense matching*, it concatenates the point features in all the semantic nodes to construct $\{\tilde{\mathbf{Z}}^A, \tilde{\mathbf{Z}}^B\}$ and run (20). They build the point features $\{\tilde{\mathbf{Z}}^A, \tilde{\mathbf{Z}}^B\}$ differently, where we illustrate their computational complexity in Table XIV.

As shown in Table XIV, dense matching requires heavier dot product operations. To verify the analysis, we select a scene to perform dense matching. The result shows that the dense matching results in a high outlier ratio and requires higher computation FLOPS than the coarse-to-fine matching. See supplementary material for details.

F. Runtime Analysis

1) Runtime in 3RScan: As shown in Table XV, our GNN and match layers are significantly faster than the baseline methods. It benefits from our sparsified scene representation. Our triplet-based GNN learns on the semantic nodes, while the geometric transformer learns on the densely distributed superpoint. So it runs much faster. This enhanced GNN speed is consistent with our FLOPS analysis in Fig. 7(b).

One limitation is that our pose estimator is relatively slower than the baseline. Their estimator runs faster because they run weighted SVD [44] on GPU. They organize the point correspondences in an array of patches. An SVD thread solves a patch of correspondences. Since the patches are gathered into a batch, solving it on GPU is very fast. On the other hand, our pose estimator handles a global set of point correspondences, which is processed on the CPU. Since the correspondences are dense, solving the MAC and the robust estimator can be relatively slow. Despite the slower runtime in pose estimation, our total registration is still faster than the baseline methods. If we only consider the network inference, our inference is three times faster than the baselines.

2) Runtime in SLAM: In the two-agent SLAM, we save our median results and rerun each step on the PyTorch platform. Then, we record the time consumption on PyTorch and compare it with HLoc. It is designed to compare the inference time in the same platform since our network is deployed by LibTorch in the SLAM system. Also, as HLoc runs offline, it ensures that we compare with HLoc fairly.

As shown in Table XVI, LightGlue is the most time-consuming step in HLoc. Global matching can also be slow when the sequence is long, as HLoc searches candidate images within the time stamp interval $[0, t_q]$. Consequently, longer sequences result in a greater number of images to search through. For instance, in a frame toward the end of the sequence, global matching can take 51.5 ms, which is twice the average time shown in Table XVI.

Compared to HLoc, our total computation time is significantly faster. As shown in Table XVI, our total processing time is considerably less than that of HLoc, even when receiving dense messages. Within our framework, the shape network and pose estimator account for the majority of the inference time due to their inputs being dense points. For Hydra's loop closure detection, it runs quite fast, including its semantic descriptor construction and similarity calculation. Those operations require low computing resources. Despite its fast inference time, Hydra detects too few loop closures and is a less competitive baseline method.

However, although our key modules run faster than HLoc, we point out that our method cannot run in real time yet. One bottleneck is the point cloud preprocessing operation, adapted from GeoTransformer [41]. It uniformly downsamples the point cloud into multiple layers of point cloud. It searches the nearby points at each layer, generating a hierarchical point cloud aggregation structure. Due to the large size of our point cloud and it runs on CPU, it can take up to 900 ms in one query frame. GeoTransformer and SG-PGM also run the same preprocessing step and take a similar time. Similar to them, we treat the preprocessing operation as a step in the data loader, which is not considered in the runtime. Besides, since data loading time in HLoc is not involved either, we exclude ours in Table XVI, which is a fair comparison. Considering that real-time performance

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

TABLE XVI
(A) REGISTRATION TIME IN THE TWO-AGENT SLAM. (B) SEMANTIC MAPPING TIME

Hydra†		**HLoc**		**SG-Reg**		
Steps	Time(ms)	Steps	Time(ms)	Steps	Coarse(ms)	Dense(ms)
Object&DBoW	0.5	NetVLAD	18.9	Shape Network	-	38.2
Global Match	14.3	Global Match	23.5	T-GNN	2.1	2.1
ORB Extract	23.8	SuperPoint	15.4	Node Match	1.5	2.0
Brutal Match	0.9	LightGlue	173.6	Point Match	-	22.4
RANSAC-PnP	10.3	RANSAC-PnP	15.6†	Pose Estimator	12.1†	43.9†
Total	49.6	**Total**	247	**Total**	15.7	108.6

(a)

	Time (ms)
RAM* [18]	29.8
GroundingDINO* [19]	130.2
EfficientSAM* [70]	43.6
FM-Fusion [11]	62.8

(b)

We report the average time in each frame. Modules or method marked in † run on CPU. FM-Fusion runs in real time, while the foundation models* run offline.

is not our motivation in this article, we leave it to be better implemented in future work.

3) Mapping Runtime: We report the semantic mapping time for reference. Semantic mapping is a high-level perception function in SLAM systems. Multiple modules use it as a shared front end [51], [52]. As a result, we excluded its timing from the scene graph registration timing and reported it separately. In computing the runtime, we run all the foundation models offline and run FM-Fusion in real time. FM-Fusion loads the pregenerated results from foundation models to construct the scene graph. As shown in Table XVI(b), our semantic mapping module can run in approximately 14 Hz. With the foundation models, the system should be able to run at 4 Hz. If the SLAM system requires the semantic perception running at a slower rate [50], the overall inference speed is acceptable for a semantic SLAM system.

G. Limitations

In the two-agent SLAM benchmark, we neglect the pose drift for each agent. Since VINS-Mono [31] provides accurate odometry over short camera trajectories ($\leq$ 150 m), we assume that the scene graph and trajectory of each agent are rigid bodies. However, a desired multiagent SLAM system should account for pose drift for each agent and further reconstruct an optimal SLAM representation. To address this limitation, we believe that combining the correspondences from SG-Reg with the image matching constraints from HLoc for optimization is a promising direction. SG-Reg can provide an accurate initial alignment between agents, which can then be followed by scene graph optimization frameworks [48], [74] to correct pose drift for each agent.

Moreover, we find some failed registration in the long-tailed scenes, such as empty rooms and scenes in symmetric layout. We provide their results in the supplementary material, which can be further improved in the future.

VIII. CONCLUSION

In this work, we propose a learning-based registration method that aligns two semantic scene graphs without requiring an initial value. The neural network encodes coarse semantic node features, which are fused from their semantic label, local topology with spatial awareness, and geometric shapes. Besides, our network encodes the dense point cloud that are belong to each semantic node. Then, the network searches for correspondences at two levels: coarse semantic nodes and dense point clouds. In the optimization stage, we employ a robust pose estimator to compute the transformation based on these correspondences. Thanks to automatic data generation, we train the network in a self-supervised manner without the need for any ground-truth semantic annotations. In the evaluation, we compare our method with point cloud registration networks and semantic registration networks. Our approach achieves a higher recall rate, particularly in medium and large scenes. In addition, our GPU inference requires less than 2.2 GFLOPS, while the baseline can require up to 1314 GFLOPS. We deploy SG-Reg in a two-agent SLAM system to align the coordinates between the two agents. In real-world indoor scenes, SG-Reg significantly outperforms handcrafted descriptors in Hydra. When compared to the state-of-the-art image matching network HLoc, SG-Reg achieves a similar registration success rate. However, in frames with large viewpoint differences, SG-Reg registers the scene more accurately and robustly than HLoc. In addition, our communication module requires only 3.9% of the bandwidth that HLoc demands. On the other hand, we recognize the system demands heavy front-end modules to reconstruct the semantic maps. In the two-agent benchmark, we ignore the pose drift within each agent and register their maps as two rigid bodies. Despite these limitations, our system is generalizable, offers higher computational efficiency during inference, and communicates data using lower bandwidth.

REFERENCES

[1] H. Xu, P. Liu, X. Chen, and S. Shen, "D^2SLAM: Decentralized and distributed collaborative visual-inertial SLAM system for aerial swarm," *IEEE Trans. Robot.*, vol. 40, pp. 3445–3464, 2024.
[2] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "ORB-SLAM3: An accurate open-source library for visual, visual–inertial, and multimap SLAM," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1874–1890, Dec. 2021.
[3] P. Yinet al., "AutoMerge: A framework for map assembling and smoothing in city-scale environments," *IEEE Trans. Robot.*, vol. 39, no. 5, pp. 3686–3704, Oct. 2023.
[4] C. Toft et al., "Long-term visual localization revisited," *IEEE Trans. Pattern Anal Mach. Intell.*, vol. 44, no. 4, pp. 2074–2088, Apr. 2022.
[5] R. Arandjelovic, P. Gronat, A. Torii, T. Pajdla, and J. Sivic, "NetVLAD: CNN architecture for weakly supervised place recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 5297–5307.
[6] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperGlue: Learning feature matching with graph neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 4937–4946.
[7] P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local feature matching at light speed," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 17581–17592.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

[8] S. Yang and S. Scherer, "CubeSLAM: Monocular 3D object SLAM," *IEEE Trans. Robot.*, vol. 35, no. 4, pp. 925–938, Aug. 2019.

[9] J. McCormac, R. Clark, M. Bloesch, A. J. Davison, and S. Leutenegger, "Fusion++: Volumetric object-level SLAM," in *Proc. Int. Conf. 3D Vis.*, 2018, pp. 32–41.

[10] S. Lin, J. Wang, M. Xu, H. Zhao, and Z. Chen, "Topology aware object-level semantic mapping towards more robust loop closure," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 7041–7048, Oct. 2021.

[11] C. Liu, K. Wang, J. Shi, Z. Qiao, and S. Shen, "FM-Fusion: Instance-aware semantic mapping boosted by vision-language foundation models," *IEEE Robot. Autom. Lett.*, vol. 9, no. 3, pp. 2232–2239, Mar. 2024.

[12] S. L. Bowman, N. Atanasov, K. Daniilidis, and G. J. Pappas, "Probabilistic data association for semantic SLAM," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 1722–1729.

[13] L. Nicholson, M. Milford, and N. Sünderhauf, "QuadricSLAM: Dual quadrics from object detections as landmarks in object-oriented SLAM," *IEEE Robot. Autom. Lett.*, vol. 4, no. 1, pp. 1–8, Jan. 2019.

[14] C. D. Don, C. D. Don, R. Siegwart, J. Nieto, and C. Cadena, "X-view: Graph-based semantic multi-view localization," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1687–1694, Jul. 2018.

[15] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3D scene graph construction and optimization," in *Proc. Robot.: Sci. Syst. Conf.*, 2022.

[16] S. D. Sarkar, O. Miksik, M. Pollefeys, D. Barath, and I. Armeni, "SGAligner: 3D scene alignment with scene graphs," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 21927–21937.

[17] Y. Xie, A. Pagani, and D. Stricker, "SG-PGM: Partial graph matching network with semantic geometric fusion for 3D scene graph alignment and its downstream tasks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 28401–28411.

[18] Y. Zhang et al., "Recognize anything: A strong image tagging model," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 1724–1732.

[19] S. Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2024, pp. 38–55.

[20] A. Kirillov et al., "Segment anything," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2023, pp. 4015–4026.

[21] K. He, G. Gkioxari, P. Dollár, and R. Girshick, "Mask R-CNN," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2961–2969.

[22] J. McCormac, A. Handa, A. Davison, and S. Leutenegger, "SemanticFusion: Dense 3D semantic mapping with convolutional neural networks," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2017, pp. 4628–4635.

[23] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, "Kimera: An open-source library for real-time metric-semantic localization and mapping," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2020, pp. 1689–1696.

[24] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Ch. Assoc. Comput. Linguistics: Hum. Lang. Technol.*, Jun. 2019, pp. 4171–4186.

[25] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?," in *Proc. Int. Conf. Learn. Represent.*, 2022.

[26] M. Cuturi, "Sinkhorn distances: Lightspeed computation of optimal transport," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2013, vol. 26, pp. 2292–2300.

[27] J. G. Mangelson, D. Dominic, R. M. Eustice, and R. Vasudevan, "Pairwise consistent measurement set maximization for robust multi-robot map merging," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2018, pp. 2916–2923.

[28] Z. Qiao, Z. Yu, B. Jiang, H. Yin, and S. Shen, "G3Reg: Pyramid graph-based global registration using Gaussian ellipsoid model," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 3416–3432, 2025.

[29] J. Wald, A. Avetisyan, N. Navab, F. Tombari, and M. Nießner, "RIO: 3D object instance re-localization in changing indoor environments," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 7657–7666.

[30] P.-E. Sarlin, C. Cadena, R. Siegwart, and M. Dymczyk, "From coarse to fine: Robust hierarchical localization at large scale," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 12708–12717.

[31] T. Qin, P. Li, and S. Shen, "VINS-Mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.

[32] D. Gálvez-López and J. D. Tardos, "Bags of binary words for fast place recognition in image sequences," *IEEE Trans. Robot.*, vol. 28, no. 5, pp. 1188–1197, Oct. 2012.

[33] E. Brachmann et al., "DSAC-differentiable RANSAC for camera localization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 6684–6692.

[34] P.-E. Sarlin et al., "LaMAR: Benchmarking localization and mapping for augmented reality," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 686–704.

[35] A. Dai, M. Nießner, M. Zollhöfer, S. Izadi, and C. Theobalt, "BundleFusion: Real-time globally consistent 3D reconstruction using on-the-fly surface reintegration," *ACM Trans. Graph.*, vol. 36, no. 4, pp. 1–18, 2017.

[36] Q.-Y. Zhou, J. Park, and V. Koltun, "Fast global registration," in *Proc. Eur. Conf. Comput. Vis.*, 2016, pp. 766–782.

[37] R. B. Rusu, N. Blodow, and M. Beetz, "Fast point feature histograms (FPFH) for 3 D registration," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2009, pp. 3212–3217.

[38] H. Yang, J. Shi, and L. Carlone, "TEASER: Fast and certifiable point cloud registration," *IEEE Trans. Robot.*, vol. 37, no. 2, pp. 314–333, Apr. 2021.

[39] Y. Wang and J. M. Solomon, "Deep closest point: Learning representations for point cloud registration," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 3523–3532.

[40] C. Choy, W. Dong, and V. Koltun, "Deep global registration," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2020, pp. 2514–2523.

[41] Z. Qin, H. Yu, C. Wang, Y. Guo, Y. Peng, and K. Xu, "Geometric transformer for fast and robust point cloud registration," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 11143–11152.

[42] H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas, "KPConv: Flexible and deformable convolution for point clouds," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 6411–6420.

[43] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep learning on point sets for 3D classification and segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 652–660.

[44] P. J. Besl and N. D. McKay, "Method for registration of 3D shapes," *Proc. SPIE*, vol. 1611, pp. 586–606, 1992.

[45] A. Zeng, S. Song, M. Nießner, M. Fisher, J. Xiao, and T. Funkhouser, "3DMatch: Learning local geometric descriptors from RGB-D reconstructions," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 1802–1811.

[46] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[47] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3D Euclidean signed distance fields for on-board MAV planning," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, Sep. 2017, pp. 1366–1373.

[48] H. Bavle, J. L. Sanchez-Lopez, M. Shaheer, J. Civera, and H. Voos, "Situational graphs for robot navigation in structured indoor environments," *IEEE Robot. Autom. Lett.*, vol. 7, no. 4, pp. 9107–9114, Oct. 2022.

[49] H. Bavle, J. L. Sanchez-Lopez, M. Shaheer, J. Civera, and H. Voos, "S-Graphs+: Real-time localization and mapping leveraging hierarchical representations," *IEEE Robot. Autom. Lett.*, vol. 8, no. 8, pp. 4927–4934, Aug. 2023.

[50] N. Hughes et al., "Foundations of spatial perception for robotics: Hierarchical representations and real-time systems," *Intl. J. Robot. Res.*, vol. 43, 2024, Art. no. 02783649241229725.

[51] A. Werby, C. Huang, M. Büchner, A. Valada, and W. Burgard, "Hierarchical open-vocabulary 3D scene graphs for language-grounded robot navigation," in *Proc. Robot.: Sci. Syst. Conf.*, 2024.

[52] Q. Gu et al., "ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning," in *Proc. the IEEE Int. Conf. Robot. Autom.*, 2023, pp. 5021–5028.

[53] X. Liu et al., "SlideSLAM: Sparse, lightweight, decentralized metric-semantic SLAM for multi-robot navigation," 2024, *arXiv:2406.17249*.

[54] L. Zhu, S. Huang, K. Schindler, and I. Armeni, "Living scenes: Multi-object relocalization and reconstruction in changing 3D environments," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 28014–28024.

[55] C. Liu and S. Shen, "Towards view-invariant and accurate loop detection based on scene graph," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 2127–2133.

[56] Z. Qian, J. Fu, and J. Xiao, "Towards accurate loop closure detection in semantic SLAM with 3D semantic covisibility graphs," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 2455–2462, Apr. 2022.

[57] J. Kabalar, S.-C. Wu, J. Wald, K. Tateno, N. Navab, and F. Tombari, "Towards long-term retrieval-based visual localization in indoor environments with changes," *IEEE Robot. Autom. Lett.*, vol. 8, no. 4, pp. 1975–1982, Apr. 2023.

[58] R. Wang, J. Yan, and X. Yang, "Neural graph matching network: Learning lawler's quadratic assignment problem with extension to hypergraph and multiple-graph matching," *IEEE Trans. Pattern Anal Mach. Intell.*, vol. 44, no. 9, pp. 5261–5279, Sep. 2022.

[59] W. E. Lorensen and H. E. Cline, "Marching cubes: A high resolution 3D surface construction algorithm," *ACM SIGGRAPH Comput. Graph.*, vol. 21, no. 4, pp. 163–169, 1987.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.

[60] J. O'Rourke, "Finding minimal enclosing boxes," *Int. J. Comput. Inf. Sci.*, vol. 14, pp. 183–199, 1985.
[61] H. Zhang et al., "DINO: DETR with improved denoising anchor boxes for end-to-end object detection," in *Proc. Int. Conf. Learn. Represent.*, 2022.
[62] B. Jiang, Y. Zhu, and M. Liu, "A triangle feature based map-to-map matching and loop closure for 2D graph SLAM," in *Proc. Int. Conf. Robot. Biomimetics*, 2019, pp. 2719–2725.
[63] C. Yuan, J. Lin, Z. Zou, X. Hong, and F. Zhang, "STD: Stable triangle descriptor for 3D place recognition," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023, pp. 1897–1903.
[64] Z. Zou et al., "LTA-OM: Long-term association LiDAR–IMU odometry and mapping," *J. Field Robot.*, vol. 41, pp. 2455–2474, 2024.
[65] C. Yuan, J. Lin, Z. Liu, H. Wei, X. Hong, and F. Zhang, "BTC: A binary and triangle combined descriptor for 3D place recognition," *IEEE Trans. Robot.*, vol. 40, pp. 1580–1599, 2024.
[66] J. Munkres, "Algorithms for the assignment and transportation problems," *J. Soc. Ind. Appl. Math.*, vol. 5, no. 1, pp. 32–38, 1957.
[67] S. Xie, J. Gu, D. Guo, C. R. Qi, L. Guibas, and O. Litany, "PointContrast: Unsupervised pre-training for 3D point cloud understanding," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 574–591.
[68] H. Yang, P. Antonante, V. Tzoumas, and L. Carlone, "Graduated non-convexity for robust spatial perception: From non-minimal solvers to global outlier rejection," *IEEE Robot. Autom. Lett.*, vol. 5, no. 2, pp. 1127–1134, Apr. 2020.
[69] A. P. Bustos, T.-J. Chin, F. Neumann, T. Friedrich, and M. Katzmann, "A practical maximum clique algorithm for matching with pairwise constraints," 2019, *arXiv:1902.01534*.
[70] Y. Xiong et al., "EfficientSAM: Leveraged masked image pretraining for efficient segment anything," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 16111–16121.
[71] A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner, "ScanNet: Richly-annotated 3 D reconstructions of indoor scenes," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2432–2443.
[72] A. Segal, D. Haehnel, and S. Thrun, "Generalized-ICP," in *Proc. Robot.: Sci. Syst. Conf.*, Seattle, WA, USA, 2009, vol. 2, pp. 435–442.
[73] L. Wang, H. Xu, Y. Zhang, and S. Shen, "Neither fast nor slow: How to fly through narrow tunnels," *IEEE Robot. Autom. Lett.*, vol. 7, no. 2, pp. 5489–5496, Apr. 2022.
[74] Y. Chang, N. Hughes, A. Ray, and L. Carlone, "Hydra-Multi: Collaborative online construction of 3D scene graphs with multi-robot teams," in *Proc. IEEE/RSJ Int. Conf. Intell. Robot. Syst.*, 2023, pp. 10995–11002.
[75] Y. Tian, Y. Chang, F. H. Arias, C. Nieto-Granda, J. P. How, and L. Carlone, "Kimera-Multi: Robust, distributed, dense metric-semantic SLAM for multi-robot systems," *IEEE Trans. Robot.*, vol. 38, no. 4, pp. 2022–2038, Aug. 2022.



Chuhao Liu received the B.Eng. degree in electrical and electronic engineering from the University of Nottingham, Nottingham, U.K., in 2013, and the M.Phil. degree in technology leadership and entrepreneurship in 2020 from the Hong Kong University of Science and Technology, Hong Kong, where he is currently working toward the Ph.D. degree in electronic and computer engineering.

His research interests include spatial perception, localization, and mapping.



Zhijian Qiao received the B.Eng. degree in automation from Northeastern University, Shenyang, China, in 2019, and the M.S. degree in control engineering from Shanghai Jiao Tong University, Shanghai, China, in 2022. He is currently working toward the Ph.D. degree with the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong.

His doctoral research focuses on robotics and autonomous driving, with specialized works in robust state estimation and crowd-sourced mapping.



Jieqi Shi received the B.Eng. degree in computer science and technology from the School of Electronics Engineering and Computer Science, Peking University, Beijing, China, in 2018, and the Ph.D. degree in electronic and computer engineering from the Hong Kong University of Science and Technology, Hong Kong, in 2024.

She is currently an Assistant Professor with the School of Intelligence Science and Technology, Nanjing University, Nanjing, China. Her research interests include robotics, with a focus on perception, localization and mapping, and learning-based autonomous navigation.



Ke Wang received the B.Eng. degree in computer science and technology from the School of Computer Science and Technology, Northwestern Polytechnical University, Xi'an, China, in 2013, and the Ph.D. degree in electronic and computer engineering from the Hong Kong University of Science and Technology, Hong Kong, in 2023.

He is currently an Assistant Professor with the School of Information Engineering, Chang'an University, Xi'an. His research interests include robotics, with a focus on localization and mapping, visual depth estimation, and vehicle-to-everything cooperation perception.



Peize Liu received the B.Sc. degree in software engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2022. He is currently working toward the Ph.D. degree with the Hong Kong University of Science and Technology, Hong Kong, under the supervision of Prof. Shaojie Shen.

His doctoral research focuses on unmanned aerial vehicles, state estimation in aerial swarm, and swarm systems.



Shaojie Shen received the B.Eng. degree in electronic engineering from the Hong Kong University of Science and Technology, Hong Kong, in 2009, and the M.S. degree in robotics and the Ph.D. degree in electrical and systems engineering from the University of Pennsylvania, Philadelphia, PA, USA, in 2011 and 2014, respectively.

In September 2014, he joined the Department of Electronic and Computer Engineering, Hong Kong University of Science and Technology, Hong Kong, as an Assistant Professor and became an Associate Professor in July 2020. His research interests include robotics and unmanned aerial vehicles, with a focus on state estimation, sensor fusion, computer vision, localization and mapping, and autonomous navigation in complex environments.

Authorized licensed use limited to: Technion Israel Institute of Technology. Downloaded on September 11,2026 at 02:01:37 UTC from IEEE Xplore. Restrictions apply.